ChatGPT

Explain me what is API with simple real time example, also give me structure/syntax of API, what are the topics need to be learnt in API, also write an API in java and springboot - and explain me line by line - remeber that I don't have any idea about API, so teach me from basic to pro, so that I should be able to write a code for API

An **API** (Application Programming Interface) is a way for **two software systems to talk to each other**.

Think of it like a **restaurant waiter**:

- You (**client app**) ask for food: "Give me a pizza"
- Waiter (**API**) takes your request to the kitchen (**server / database**)
- Kitchen prepares it and gives it back through the waiter
- You receive the pizza (**response data**)

A simple real-time example (super relatable)

Example: Weather app

Your phone's weather app does **not** "magically know" the weather.

It calls a weather **API** like:

- "Give me weather for Mumbai"
- API responds:
  - temperature: 32
  - humidity: 70
  - condition: "Cloudy"

So:
Weather app → sends request → Weather API → returns response → app shows weather

---

API structure / syntax (what actually happens)

Most beginner-friendly APIs are **REST APIs** over **HTTP**.

1) Request format

A request usually has:

(A) URL (Endpoint)

- Example: `https://api.example.com/weather`

(B) Method (Action)

- `GET` → read data
- `POST` → create data
- `PUT` → update full data
- `PATCH` → update partial data
- `DELETE` → remove data

(C) Headers (Metadata)

- Example: content type, auth token
- `Content-Type: application/json`
- `Authorization: Bearer <token>`

(D) Body (Data you send)

Only mainly for `POST/PUT/PATCH`.

Example body (JSON):

JSON

```
{
  "city": "Mumbai"
}
```

---

2) Response format

Response has:

(A) Status code

- `200 OK` → success
- `201 Created` → created successfully
- `400 Bad Request` → you sent wrong input
- `401 Unauthorized` → no/invalid login token
- `404 Not Found` → endpoint/data not found
- `500 Internal Server Error` → server crashed

(B) Response body (JSON usually)

JSON

```
{
  "city": "Mumbai",
  "temp": 32,
  "condition": "Cloudy"
}
```

API topics you should learn (basic → pro roadmap)

Beginner (must know)

1. **What is client & server**
2. **HTTP basics** (URL, request, response)
3. **HTTP methods** (GET/POST/PUT/PATCH/DELETE)
4. **Status codes** (200, 201, 400, 401, 404, 500)
5. **JSON** (how data is sent/received)
6. **REST basics** (resources like `/users`, `/products`)

Intermediate

7. **Spring Boot basics**
8. **Controllers** (`@RestController`)
9. **DTOs / Request & Response objects**
10. **Validation** (`@NotNull`, `@Size`)
11. **Exception handling** (`@ControllerAdvice`)
12. **Logging**
13. **Testing APIs** using Postman / curl

Pro level

14. **Authentication & Authorization**
    - JWT, OAuth2
15. **Database + JPA**
    - Entities, repositories, service layer
16. **API Documentation**
    - Swagger / OpenAPI
17. **Pagination, Sorting, Filtering**
18. **Versioning**
    - `/api/v1/...`
19. **Security Best Practices**

20. **Performance**
- caching, rate limiting

21. **Microservices basics** (later)

Now: Let's write a real API in Java + Spring Boot (Beginner-friendly)

We'll build a simple **Student API**:

- `GET /students` → returns all students
- `GET /students/{id}` → returns one student
- `POST /students` → adds a new student

No database yet (we'll store in memory). This is perfect for learning.

Project structure (typical Spring Boot API)

```
src/main/java/com/example/demo
  DemoApplication.java
  controller/StudentController.java
  model/Student.java
  service/StudentService.java
```

---

1) Student model class (data structure)

Java

```java
package com.example.demo.model;

public class Student {
    private int id;
    private String name;
    private int age;

    public Student() { }

    public Student(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }
```

```
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
```

Line-by-line (simple meaning)

- `package ...` → tells Java which folder/group this file belongs to
- `public class Student` → we're creating a "Student" type
- `private int id;` → student id field
- `private String name;` → student name field
- `private int age;` → student age field
- `public Student()` → empty constructor (Spring needs it sometimes)
- `public Student(int id, String name, int age)` → constructor to create Student quickly
- `getX()` → used to read values
- `setX()` → used to set/update values

---

2) StudentService (business logic layer)

Java

```java
package com.example.demo.service;

import com.example.demo.model.Student;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Service
public class StudentService {

    private final List<Student> students = new ArrayList<>();

    public StudentService() {
        students.add(new Student(1, "Asha", 20));
        students.add(new Student(2, "Ravi", 22));
    }

    public List<Student> getAllStudents() {
        return students;
    }
```

Line-by-line explanation

- `@Service` → tells Spring "this class has business logic"
- `private final List<Student> students...` → in-memory list acting like a database
- `public StudentService()` → constructor adds 2 sample students
- `getAllStudents()` → returns list
- `getStudentById(int id)` → searches list and returns student if found
- `Optional<Student>` → means "maybe student exists, maybe not"
- `addStudent(Student student)` → adds student to list and returns it

---

3) StudentController (API endpoints)

Java

```java
package com.example.demo.controller;

import com.example.demo.model.Student;
import com.example.demo.service.StudentService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
```

Line-by-line (very clear)

- `@RestController` → "this class provides APIs and returns JSON"
- `@RequestMapping("/students")` → base URL becomes `/students`
- `private final StudentService studentService;` → controller uses service
- `public StudentController(...)` → Spring injects (gives) the service automatically
- `@GetMapping` → handles `GET /students`
- `getAllStudents()` → returns list as JSON automatically

- `@GetMapping("/{id}")` → handles `GET /students/2`
- `@PathVariable int id` → reads `2` from URL
- `ResponseEntity` → lets us control status codes like `404`
- If student found → `200 OK`
- If not found → `404 Not Found`
- `@PostMapping` → handles `POST /students`
- `@RequestBody Student student` → reads JSON body and converts into Student object
- returns saved student as response

How to test your API (using Postman or curl)

1) Get all students

GET `http://localhost:8080/students`

Response:

```json
[
  {"id":1,"name":"Asha","age":20},
  {"id":2,"name":"Ravi","age":22}
]
```

2) Get student by id

GET `http://localhost:8080/students/2`

3) Add a student

POST `http://localhost:8080/students`

Body (JSON):

```json
{
  "id": 3,
  "name": "Kiran",
  "age": 21
}
```

What to learn next (so you become “pro” fast)

If you want, I can teach this step-by-step in the best order:

1. Add **validation** (reject missing name/age)
2. Add **proper status code** `201 Created` for POST
3. Add **DELETE** and **PUT**
4. Connect to **database** (MySQL) using Spring Data JPA
5. Add **Swagger** documentation
6. Add **JWT security**

If you tell me **you are using IntelliJ or Eclipse**, I can give you the exact steps to create the Spring Boot project + dependencies, and we’ll build it like a mini course.

I am using Eclipse

Cool — with **Eclipse**, you can build Spring Boot APIs in a very clean way. Here’s a complete beginner-friendly path: **create project** → **run** → **test** → **understand files**.

1) Install what you need in Eclipse (once)

1. Open **Eclipse**
2. Go to **Help** → **Eclipse Marketplace**
3. Search and install:
   - **Spring Tools 4 (aka Spring Tools / STS)**
4. Restart Eclipse

This gives you “Spring Starter Project” option inside Eclipse.

---

2) Create a Spring Boot project in Eclipse

1. **File** → **New** → **Other…**
2. Choose **Spring Boot** → **Spring Starter Project**
3. Fill:
   - **Name:** `student-api`
   - **Type:** Maven
   - **Java:** 17 (or 11 if that’s what you have)

- **Packaging:** Jar
- **Group:** `com.example`
- **Artifact:** `student-api`
- **Package:** `com.example.studentapi`

4. Click **Next**
5. Select dependencies:
   - ✅ **Spring Web**
6. Finish

Eclipse will download dependencies (first time can take a bit).

---

3) Run the project

1. In Project Explorer, open:

   `src/main/java → com.example.studentapi → StudentApiApplication.java`
2. Right click → **Run As** → **Spring Boot App**
3. In Console you should see something like:

   `Tomcat started on port(s): 8080`

Your server is running at:

`http://localhost:8080`

---

4) Now add the API code (copy-paste + understand)

A) Create packages

Inside `src/main/java/com/example/studentapi/` create 3 packages:

- `controller`
- `service`
- `model`

Right click your base package → **New** → **Package**

B) Create `Student.java`

Path: `model/Student.java`

Java

```java
package com.example.studentapi.model;

public class Student {
    private int id;
    private String name;
    private int age;

    public Student() { }

    public Student(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }
```

What this is

This is a **data class**. When your API returns JSON, Spring converts a `Student` object into JSON automatically.

C) Create `StudentService.java`

Path: `service/StudentService.java`

Java

```java
package com.example.studentapi.service;

import com.example.studentapi.model.Student;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Service
public class StudentService {

    private final List<Student> students = new ArrayList<>();

    public StudentService() {
        students.add(new Student(1, "Asha", 20));
```

```
        students.add(new Student(2, "Ravi", 22));
    }

    public List<Student> getAllStudents() {
        return students;
    }
```

Line-by-line (simple)

- `@Service` → Spring manages this class (creates its object automatically)
- `students` list → temporary "database" in memory
- constructor adds sample students
- `getAllStudents()` → list return
- `getStudentById()` → search by id
- `addStudent()` → add new student

D) Create `StudentController.java`

Path: `controller/StudentController.java`

Java

```java
package com.example.studentapi.controller;

import com.example.studentapi.model.Student;
import com.example.studentapi.service.StudentService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
```

Line-by-line (beginner)

- `@RestController` → "This class contains APIs and returns JSON"

- `@RequestMapping("/students")` → base URL is `/students`
- `StudentService` injected into controller (Spring gives it automatically)
- `@GetMapping` → handles **GET /students**
- `@GetMapping("/{id}")` → handles **GET /students/2**
- `@PathVariable` → gets `2` from URL
- `ResponseEntity` → allows returning `404` if not found
- `@PostMapping` → handles **POST /students**
- `@RequestBody` → reads JSON and converts it into `Student` object

---

5) Test your API (easy)

Option A: Browser test (GET)

Open in browser:

- `http://localhost:8080/students`
- `http://localhost:8080/students/1`

Option B: Postman (recommended for POST)

GET

- Method: GET
- URL: `http://localhost:8080/students`

POST

- Method: POST
- URL: `http://localhost:8080/students`
- Body → raw → JSON:

JSON

```json
{
  "id": 3,
  "name": "Kiran",
  "age": 21
}
```

Then do GET again and you'll see the new student.

---

6) Common Eclipse issues (quick fixes)

- **Port 8080 already used**
  - Stop other app OR change port in `src/main/resources/application.properties`:

```
server.port=8081
```

- **Maven dependencies not downloaded**
  - Right click project → **Maven** → **Update Project**
- **Java version mismatch**
  - Project → Properties → Java Compiler → set version
  - Also ensure correct JDK in Eclipse preferences

Next step (to become "pro")

If you follow me step-by-step, next we'll upgrade this API professionally:

1. Add **DELETE /students/{id}**
2. Add **PUT /students/{id}**
3. Add **validation** (name required, age range)
4. Add **database (MySQL)** with Spring Data JPA
5. Add **Swagger UI**
6. Add **JWT security**

If you want, tell me:

- Which Java version you have (11/17)?
- Are you able to run the app and open `http://localhost:8080/students`?

hold on, java and spring boot is best or dotnet is best for writing APIs, as a beginner which is easy to learn, if I learn java and spring boot, will it be easy for me to learn dot net?

Neither is "best" for everyone — **both Java + Spring Boot and .NET (C# + ASP.NET Core) are excellent for APIs**. The right pick depends on your goals and what environment you're in.

Which is easier for a beginner?

Most beginners find **.NET (C# + ASP.NET Core)** a bit easier to start with because:

- tooling is very smooth (especially with Visual Studio),
- the framework feels more "guided" out of the box,
- fewer concepts hit you at once.

Spring Boot is also beginner-friendly, but you'll bump into:

- more annotations,
- more "Spring magic" (dependency injection, configuration),
- and sometimes more setup choices.

If you're staying on Eclipse, Spring Boot is totally fine — just slightly more "framework concepts" early on.

Which is better for jobs?

Both are in high demand. A simple way to decide:

- If you see more **Java/Spring** roles around you (or your company uses Java) → go **Java + Spring Boot**
- If you see more **.NET** roles (or Microsoft stack, Azure-heavy shops) → go **.NET**

What's "better" technically for APIs?

For REST APIs, both give you:

- fast performance
- clean routing (`/students/{id}`)
- JSON serialization
- validation
- database support (ORM)
- security (JWT/OAuth)
- OpenAPI/Swagger
- testing support

In real projects, the "best" is usually the one your team already uses.

If I learn Java + Spring Boot, will .NET be easy later?

Yes — **very**. Once you understand API fundamentals, switching is mostly syntax + framework conventions.

Concepts that transfer 1:1

- HTTP methods: GET/POST/PUT/DELETE
- status codes: 200/201/400/401/404/500
- request/response JSON
- controllers and routes
- dependency injection
- service layer pattern

- DTOs
- validation
- exception handling
- database + ORM (JPA vs EF Core)
- auth (JWT/OAuth)
- Swagger/OpenAPI

Quick mapping (Spring → .NET)

- `@RestController` → `[ApiController]`
- `@GetMapping("/x")` → `[HttpGet("x")]`
- `@PostMapping` → `[HttpPost]`
- `@RequestBody` → method parameter from body (`[FromBody]`)
- `ResponseEntity` → `ActionResult` / `IActionResult`
- Spring DI (`@Service`) → .NET DI (`services.AddScoped<...>()`)

So if you learn one properly, the other becomes mostly "learning the new syntax and project structure".

My recommendation as a beginner

Pick based on your situation:

Choose Java + Spring Boot if:

- you already started it (you did)
- your environment/company uses Java
- you want strong backend fundamentals and enterprise stack exposure

Choose .NET if:

- you want the easiest "onboarding" experience
- you can use Visual Studio (Windows is very smooth)
- you're aiming for Microsoft/Azure-heavy roles

If you're already set up in Eclipse and learning Spring Boot now: **stick with it**. Don't switch too early; getting good at one stack is more valuable than switching repeatedly.

If you want, tell me your goal:

- "job in X months"
- your OS (Windows/macOS/Linux)
- whether you prefer Eclipse only or can use IntelliJ/VS Code/Visual Studio
  …and I'll recommend a best learning path (and mini-project) for whichever stack you choose.

2. What are the related XML technologies that is used to create and consume XML based Web Services?

Answer it

Related XML technologies used to create and consume XML-based web services include:

- **XML** — the core format used to structure data
- **XSD (XML Schema Definition)** — defines the rules and structure of XML data
- **WSDL (Web Services Description Language)** — describes the web service, its methods, inputs, and outputs
- **SOAP (Simple Object Access Protocol)** — protocol for exchanging XML messages in web services
- **UDDI (Universal Description, Discovery, and Integration)** — used to publish and discover web services

These technologies work together in XML-based web services.

5. In Rest architecture everything is treated as _____ and it can be accessed by a _____

In REST architecture, everything is treated as a **resource** and it can be accessed by a **URI (Uniform Resource Identifier)**.

How to create Maven project in eclipse give me step by step

Here's the step-by-step way to create a **Maven project in Eclipse**.

Method 1: Create a normal Maven project

Step 1: Open Eclipse

Start Eclipse and wait for the workspace to load.

Step 2: Go to New Project

Click:
File → New → Other

Step 3: Choose Maven Project

In the search box, type **Maven**
Select:
Maven → Maven Project
Click **Next**

Step 4: Select workspace location

Usually keep:

- **Create a simple project** unchecked first
- default workspace location as it is

Click **Next**

Step 5: Select Archetype

Now Eclipse asks for an **archetype**.

An archetype is like a project template.

For a basic Java Maven project, choose:

- **maven-archetype-quickstart**

Group Id usually:

- `org.apache.maven.archetypes`

Artifact Id:

- `maven-archetype-quickstart`

Select it and click **Next**

Step 6: Enter project details

Fill these values:

- **Group Id**: `com.example`
- **Artifact Id**: `myapp`
- **Version**: keep default

- **Package**: `com.example`

Then click **Finish**

Step 7: Wait for project creation

Eclipse will download Maven dependencies from the internet.
This may take some time the first time.

What you will see

Your Maven project will be created with structure like:

```
myapp
 ├── src/main/java
 ├── src/test/java
 └── pom.xml
```

Important file:

- **pom.xml** → this is the main Maven file where dependencies are added

---

Method 2: Create a simple empty Maven project

If you do not want archetype sample code:

Step 1

File → New → Other → Maven Project

Step 2

Click **Next**

Step 3

Check:
Create a simple project (skip archetype selection)

Click **Next**

Step 4

Enter:

- **Group Id**: `com.example`
- **Artifact Id**: `demo`

- **Packaging**: `jar`

Click **Finish**

This creates a cleaner Maven project.

If Maven option is not visible in Eclipse

You may need Maven plugin.

Check this:

Go to:
Help → Eclipse Marketplace

Search for:

- **Maven Integration for Eclipse**
- usually called **m2e**

Install it and restart Eclipse.

How to add dependencies in Maven

Open `pom.xml` and inside `<dependencies>` add dependency code.

Example:

XML

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
        <version>3.3.2</version>
    </dependency>
</dependencies>
```

Then save the file. Maven will download it automatically.

How to update Maven project

If dependencies do not download:

- Right click project

- **Maven → Update Project**
- Click **OK**

Common issues

1. Red error in project

Try:

- Right click project → **Maven** → **Update Project**
- Then:
- **Project** → **Clean**

2. Java version problem

Check installed JDK and set correct version in:

- **Window** → **Preferences** → **Java** → **Installed JREs**

3. Internet/dependency download issue

Make sure internet is working because Maven downloads libraries online.

Best choice for you

For learning APIs and Spring Boot:

- use **Spring Starter Project** for Spring Boot apps
- use **Maven Project** for normal Java Maven apps

So:

- **plain Java project** → Maven Project
- **Spring Boot API project** → Spring Starter Project

Very short version

Path is:

File → New → Other → Maven Project → Next → choose archetype or simple project → enter Group Id and Artifact Id → Finish

If you want, I can give you:
"How to create Spring Boot Maven project in Eclipse step by step with screenshots-style explanation"

provide fix

Your `pom.xml` has **invalid XML**, so Maven cannot read it.

From the screenshot, the problem is this line:

```xml
<!-- Source : https://mvnrepository.com/artifact/org.springframework.boot/
spring-boot-starter-web-->
```

The error happens because the **comment is not written safely** around that area, or your `<dependency>` tag is placed in the wrong spot due to broken XML structure.

Fix

Open `pom.xml` and replace it with this clean working version:

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.Wipro</groupId>
    <artifactId>Myapp</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
            <version>3.3.2</version>
        </dependency>
    </dependencies>
```

```
</project>
```

Important

Do **not** put random text outside tags.
If you want to keep a comment, write it like this:

```XML
<!-- Source: mvnrepository spring-boot-starter-web -->
```

Do not break the comment badly, and do not place text in between XML tags incorrectly.

After fixing

Do these steps in Eclipse:

1. Save `pom.xml`
2. Right click project
3. Click **Maven** → **Update Project**
4. Click **OK**
5. If error still shows:
   - **Project** → **Clean**
   - then refresh project

One more important thing

Your project is showing:

```
JRE System Library [JavaSE-1.8]
```

But `spring-boot-starter-web` version `3.3.2` needs **Java 17+**.

So either:

Option 1: Use Java 17

Best option for Spring Boot 3.

Option 2: Stay on Java 8

Then use older Spring Boot version like:

XML

```
<version>2.7.18</version>
```

If you are on Java 8 right now, use this full `pom.xml` instead:

XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.Wipro</groupId>
    <artifactId>Myapp</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
            <version>2.7.18</version>
        </dependency>
    </dependencies>

</project>
```

Best recommendation

Since you are learning APIs, use:

- **Java 17**
- **Spring Boot 3**

If you want, paste your full `pom.xml` here and I'll correct it exactly line by line.

its a code for API, convert this images into code to copy paste also correct if any mistakes are there and highlight it, then teach me what is the use of it line by line

Yes — your screenshots are of a `pom.xml` file, and there are a few mistakes in it.

The biggest issue is: you have added some `<dependency>` blocks **outside** the `<project>` tag. That makes the XML invalid, so Maven fails.

Correct copy-paste version

Use this full `pom.xml`:

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.ap

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.wipro</groupId>
    <artifactId>HelloServiceAPI</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <name>HelloServiceAPI</name>
    <url>http://www.example.com</url>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
    </properties>

    <dependencies>
        <dependency>
```

What was wrong in your file

1. Extra dependencies were written after `</project>`

In your screenshots, after this:

```XML
</project>
```

you again added:

```XML
<dependency>
...
</dependency>
```

That is invalid.

Wrong

```XML
</project>

<dependency>
    ...
</dependency>
```

Correct

All dependencies must be **inside**:

```XML
<dependencies>
    ...
</dependencies>
```

and all of that must be **inside** `<project> ... </project>`.

---

2. You added `spring-boot-maven-plugin` as dependency in one place

This is wrong:

XML

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <version>4.1.0-M2</version>
</dependency>
```

Why wrong?

Because `spring-boot-maven-plugin` is **not a normal dependency**.

It must go inside the `<build><plugins>` section, not inside `<dependencies>`.

---

3. You used milestone version `4.1.0-M2`

This is not good for beginners.

Why?

- it is a milestone / pre-release version
- may be unstable
- harder to debug

Use stable version like:

XML

```xml
3.4.4
```

---

4. Your comment blocks were messy

These are okay only if written safely:

XML

```xml
<!-- Source: mvnrepository link -->
```

But if comment placement breaks XML structure, Maven throws parsing error.

For now, remove all source comments until your project runs.

Best version for Spring Boot beginners

Even better than the above, use **Spring Boot parent**. This is the standard way.

Copy-paste this version instead:

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.ap

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/>
    </parent>

    <groupId>com.wipro</groupId>
    <artifactId>HelloServiceAPI</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>HelloServiceAPI</name>
    <description>Simple Spring Boot API project</description>

    <properties>
```

Why this version is better

Because:

- you do not need to write versions for every Spring Boot dependency
- Spring manages compatible versions automatically
- cleaner and easier for beginners

Line-by-line explanation

Now I'll teach this from basic.

Line 1

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
```

Use:

This tells the file is an **XML file** and uses **UTF-8 encoding**.

Simple meaning:

"Hey computer, read this as XML text."

Line 2 to 4

```XML
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apac
```

Use:

This starts the Maven project file and tells Maven:

- which XML format we are using
- which schema rules to follow

Simple meaning:

"This whole file is a Maven project definition."

Line 5

```XML
<modelVersion>4.0.0</modelVersion>
```

Use:

Specifies Maven POM model version.

Simple meaning:

This is the standard Maven project structure version.

Parent section

XML

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.4</version>
    <relativePath/>
</parent>
```

Use:

This tells your project to inherit default settings from Spring Boot.

Why useful:

It automatically manages:

- dependency versions
- plugin versions
- build settings

Simple meaning:

"Use Spring Boot's ready-made project setup."

Group ID

XML

```xml
<groupId>com.wipro</groupId>
```

Use:

This is your company or package group name.

Simple meaning:

It identifies who owns the project.

Usually:

- company name
- domain style
- example: `com.wipro`, `com.example`

Artifact ID

```XML
<artifactId>HelloServiceAPI</artifactId>
```

Use:

This is the project name / jar name.

Simple meaning:

This is the name of your application.

When built, jar may look like:

`HelloServiceAPI-0.0.1-SNAPSHOT.jar`

Version

```XML
<version>0.0.1-SNAPSHOT</version>
```

Use:

Version of your project.

Meaning:

- `0.0.1` → first version
- `SNAPSHOT` → still under development

Name

```XML
<name>HelloServiceAPI</name>
```

Use:

Human-readable project name.

Meaning:

Shown in tools, builds, reports.

Description

```XML
<description>Simple Spring Boot API project</description>
```

Use:

Explains what the project is.

Meaning:

Optional, but helpful.

Properties

```XML
<properties>
    <java.version>17</java.version>
</properties>
```

Use:

Stores reusable project settings.

Meaning:

This says your project uses **Java 17**.

Important:

If your Eclipse is still on **Java 8**, this will fail.
You must install and configure **JDK 17**.

Dependencies section

```XML
<dependencies>
```

Use:

This section contains all external libraries your project needs.

Simple meaning:

"These are the tools/packages my project depends on."

Web dependency

XML

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Use:

Adds Spring Boot web support.

This gives you:

- REST API support
- embedded Tomcat server
- JSON conversion
- annotations like `@RestController`, `@GetMapping`, `@PostMapping`

Simple meaning:

This is the main library for creating APIs.

Without this, you cannot easily create Spring web APIs.

Test dependency

XML

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

Use:

Adds testing libraries.

Includes tools for:

- JUnit
- Mockito
- Spring test support

`scope`

```xml
<scope>test</scope>
```

means:
this dependency is only used during testing, not while running the app.

Simple meaning:

Used for writing and running test cases.

Build section

```xml
<build>
```

Use:

Contains build-related settings.

Meaning:

Tells Maven how to build your project.

Plugins

```xml
<plugins>
```

Use:

Special tools Maven uses during build.

Spring Boot Maven plugin

```xml
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
```

Use:

Helps Maven run and package Spring Boot applications properly.

What it does:

- creates executable jar
- helps run app with Maven
- supports Spring Boot packaging

Simple meaning:

This plugin makes your Spring Boot app runnable.

Closing project

```xml
</project>
```

Use:

Ends the Maven project definition.

Very important rule to remember

Inside `pom.xml`:

- dependencies go inside `<dependencies>`
- plugins go inside `<build><plugins>`
- everything must stay inside `<project>`

Structure:

XML

```xml
<project>
   ...
   <dependencies>
      <dependency>...</dependency>
   </dependencies>

   <build>
      <plugins>
         <plugin>...</plugin>
      </plugins>
   </build>
</project>
```

What this `pom.xml` does in your API project

This file tells Maven:

- project name is `HelloServiceAPI`
- use Spring Boot
- use Java 17
- include web library to build API
- include test library for testing
- include Maven plugin to run/package app

So this `pom.xml` is like the **configuration brain** of your project.

After pasting this, do these steps in Eclipse

1. Save `pom.xml`

2. Right click project

Choose:

Maven → Update Project

3. Click OK

4. If still red:

- **Project → Clean**
- Right click project → **Refresh**

One important warning

Earlier your screenshot showed:
JavaSE-1.8

But this `pom.xml` uses:

```XML
<java.version>17</java.version>
```

So you must use **JDK 17**.

If you only have Java 8, tell me and I'll give you:

- a **Java 8 compatible Spring Boot pom.xml**
- and show how to set it up in Eclipse

Best next step

After fixing `pom.xml`, the next file you need is the main Spring Boot class, like this:

```Java
package com.wipro;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HelloServiceApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(HelloServiceApiApplication.class, args);
    }
}
```

Then after that, we create your first API controller.

Paste your current Java version or your current `module-info.java`, and I'll tell you the exact next fix.

Convert those 4 images into pom.xml file as it is without any change and send me

Here is your **exact** `pom.xml` **content converted from the images (as-is, without any correction or change):**

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apa

<modelVersion>4.0.0</modelVersion>

<groupId>com.wipro</groupId>
<artifactId>HelloServiceAPI</artifactId>
<version>0.0.1-SNAPSHOT</version>

<name>HelloServiceAPI</name>

<!-- FIXME change it to the project's website -->
<url>http://www.example.com</url>

<properties>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<maven.compiler.release>17</maven.compiler.release>
</properties>
```

If you want, next I can:

- fix this properly (so your project runs)
- or explain **why this exact version is failing in Eclipse (very important for exams/interviews)**

```
@PutMapping("/{movieId}")
public ResponseEntity<Object> updateMovie(@PathVariable("movieId")String mId,@RequestBody Movie movie)
{
    if(movies.containsKey(mId))
    {
        movies.put(mId, movie); // 200-OK
        return new ResponseEntity<Object>(HttpStatus.NO_CONTENT); // 204- NO-content
    }
    else
    {
        return new ResponseEntity<Object>(HttpStatus.INTERNAL_SERVER_ERROR); //500 - internal server error
    }
```

explain

Let's break this code **very simply, line by line**, so you clearly understand what's happening.

Your code (clean version)

Java

```java
@PutMapping("/{movieId}")
public ResponseEntity<Object> updateMovie(@PathVariable("movieId") String mId,
                                          @RequestBody Movie movie) {

    if (movies.containsKey(mId)) {
        movies.put(mId, movie); // 200-OK
        return new ResponseEntity<Object>(HttpStatus.NO_CONTENT); // 204 No-conte
    } else {
        return new ResponseEntity<Object>(HttpStatus.INTERNAL_SERVER_ERROR); // 5
    }
}
```

What this API does (big picture)

This is an API to **update an existing movie**.

- URL example:

```
PUT /movies/101
```

- Body (JSON):

JSON

```
{
  "name": "Inception",
```

```
  "rating": 9
}
```

Line-by-line explanation

---

1. Mapping the API

```java
@PutMapping("/{movieId}")
```

Meaning:

- This API will be called using **HTTP PUT**
- URL will look like:

  ```
  /movies/{movieId}
  ```

  Example:

  ```
  /movies/101
  ```

Simple:

"Call this method when user sends PUT request to update a movie"

---

2. Method declaration

```java
public ResponseEntity<Object> updateMovie(...)
```

Meaning:

- `public` → accessible API
- `ResponseEntity<Object>` → response will include:
  - status code (200, 404, etc.)
  - optional data

Simple:

"This method returns HTTP response (status + data)"

---

3. Path variable

```java
@PathVariable("movieId") String mId
```

Meaning:

- Extracts value from URL

Example:

```
PUT /movies/101
```

Then:

```java
mId = "101"
```

Simple:

"Get movie ID from URL"

---

4. Request body

```java
@RequestBody Movie movie
```

Meaning:

- Takes JSON from request body and converts into `Movie` object

Example JSON:

```json
{
  "name": "Inception",
  "rating": 9
}
```

Converted into:

```java
Movie movie
```

Simple:

"Get new movie data from user"

---

5. Check if movie exists

```java
if (movies.containsKey(mId))
```

Meaning:

- `movies` is probably a `Map<String, Movie>`
- checks if movie ID already exists

Simple:

"If movie exists, update it"

---

6. Update movie

```java
movies.put(mId, movie);
```

Meaning:

- replaces old movie data with new one

Simple:

"Update the movie"

---

7. Return success response

```java
return new ResponseEntity<Object>(HttpStatus.NO_CONTENT);
```

Meaning:

- HTTP status: **204 No Content**
- update successful, but no data returned

Simple:

"Update done successfully"

---

8. Else block (if movie not found)

```java
else {
    return new ResponseEntity<Object>(HttpStatus.INTERNAL_SERVER_ERROR);
}
```

Meaning:

- returns **500 error**

⚠️ Problem here:

This is **not correct design**

If movie not found, correct status should be:

```java
HttpStatus.NOT_FOUND (404)
```

Simple:

"This part is wrong — should return 404, not 500"

Correct version (recommended)

```java
@PutMapping("/{movieId}")
public ResponseEntity<Object> updateMovie(@PathVariable String movieId,
```

```
                                        @RequestBody Movie movie) {

    if (movies.containsKey(movieId)) {
        movies.put(movieId, movie);
        return new ResponseEntity<>(HttpStatus.NO_CONTENT); // 204
    } else {
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // 404
    }
}
```

Summary (easy memory)

- `@PutMapping` → update API
- `@PathVariable` → get ID from URL
- `@RequestBody` → get data from request
- `Map.containsKey()` → check existence
- `Map.put()` → update data
- `204` → success (no content)
- `404` → not found
- ❌ `500` → should NOT be used here

Real-life analogy

- You try to update a contact in your phone
- If contact exists → updated ✅
- If not → "Contact not found" ❌

You don't say:
"Phone crashed" (500 error) 😄

If you want next, I can:

- show you **POST (create)** API
- show **DELETE API**
- or build a **full CRUD movie API step-by-step**

how to update in post man

Good question — let's do this **step-by-step in Postman** so you can actually test your `PUT` API.

What you are doing

You are calling this API:

```java
@PutMapping("/{movieId}")
```

So in Postman, you will send:
👉 **PUT request** with:

- URL → includes movieId
- Body → new movie data (JSON)

Step-by-step in Postman

1. Open Postman

---

2. Select HTTP Method

At top left:

👉 Change from `GET` → **PUT**

---

3. Enter URL

Example:

```
http://localhost:8080/movies/101
```

Meaning:

- `movies` → your controller base path
- `101` → movieId

---

4. Go to Body tab

Click:
👉 **Body**

Then select:
👉 **raw**

Then choose:
👉 **JSON** (important)

---

5. Enter JSON data

Example:

JSON

```
{
  "name": "Inception Updated",
  "rating": 9
}
```

This will be converted into:

Java

```
Movie movie
```

---

6. Click Send

What response you should see

If movie exists:

- Status: **204 No Content**
- Body: empty

👉 Means update successful

If movie does NOT exist:

- Status: **404 Not Found** (if you fixed your code)

👉 Means movie ID not present

Important things to check

1. Server must be running

Make sure your Spring Boot app is running:

```
Tomcat started on port 8080
```

---

2. URL must match controller

If your controller has:

```java
@RequestMapping("/movies")
```

Then URL must be:

```
http://localhost:8080/movies/{id}
```

---

3. Headers (usually auto set)

Postman automatically sets:

```
Content-Type: application/json
```

If not, add manually in Headers tab.

Common mistakes (very important)

❌ Using GET instead of PUT

→ Will not update

❌ Not selecting "raw JSON"

→ API won't read body

❌ Wrong URL

→ 404 error

❌ Server not running

→ connection refused

Quick summary

Step	Action
1	Select PUT
2	Enter URL `/movies/{id}`
3	Body → raw → JSON
4	Enter JSON data
5	Click Send

Want next?

I can show you:

- POST (create movie)
- DELETE (remove movie)
- or full CRUD API with Postman collection (real practice)

Just tell 👍

1. Create a RESTful Web Service and Test its lifecycle as per the below requirements:
   a. The restful web service is available on the path "employee".
   b. The web service has only one method that returns the list of employees.
   c. The method also should display the count of the number of times this method was invoked.
   Hint: Implement Per-Request / Singleton lifecycle as per the requirement. @Singleton

JAVA & SPRINGBOOT

Below is a complete **Java + Spring Boot** solution for your requirement.

Requirement understood

You need a REST API:

- path should be `/employee`
- it should have **only one method**
- that method should return a **list of employees**
- it should also show **how many times the method was invoked**
- hint says implement **Per-Request / Singleton lifecycle**

In Spring Boot:

- **Singleton** = default scope of controller/service bean
- **Per-request** = `@RequestScope`

---

1) Best answer idea

Since the question asks to show lifecycle behavior, I'll give you:

- **Singleton version**
- **Request scope version**
- explain which one will keep the counter properly

Important concept

If you want to keep increasing invocation count across requests, **Singleton** is the correct one.

Why?

- Singleton object is created once for the whole app
- same object handles all requests

- counter value keeps increasing

If you use **Per-request**, a new object is created for every request, so counter resets each time.

---

2) Spring Boot project structure

```
src/main/java/com/example/demo
    DemoApplication.java
    controller/EmployeeController.java
    model/Employee.java
```

---

3) `DemoApplication.java`

Java

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Use

This is the main class that starts the Spring Boot application.

---

4) `Employee.java`

Java

```java
package com.example.demo.model;

public class Employee {

    private int id;
    private String name;
```

```
    private double salary;

    public Employee() {
    }

    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public int getId() {
        return id;
    }
```

Use

This is the Employee model class.
It represents one employee object.

---

5) `EmployeeController.java` — Singleton lifecycle version

Java

```java
package com.example.demo.controller;

import com.example.demo.model.Employee;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestMapping;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/employee")
public class EmployeeController {

    private int count = 0;

    @GetMapping
    public Map<String, Object> getEmployees() {
        count++;
```

---

6) Output example

When you call:

```
GET http://localhost:8080/employee
```

First call response

JSON

```json
{
  "invocationCount": 1,
  "employees": [
    {
      "id": 1,
      "name": "Asha",
      "salary": 50000.0
    },
    {
      "id": 2,
      "name": "Ravi",
      "salary": 60000.0
    },
    {
      "id": 3,
      "name": "Kiran",
      "salary": 70000.0
    }
  ]
}
```

Second call response

JSON

```json
{
  "invocationCount": 2,
  "employees": [
    {
      "id": 1,
      "name": "Asha",
      "salary": 50000.0
    },
    {
      "id": 2,
      "name": "Ravi",
      "salary": 60000.0
    },
    {
```

```
      "id": 3,
      "name": "Kiran",
      "salary": 70000.0
    }
  ]
}
```

So this proves **Singleton lifecycle**.

---

7) Line-by-line explanation of controller

Class annotations

```java
@RestController
```

Means this class is a REST controller and returns JSON response.

```java
@RequestMapping("/employee")
```

Means base URL is `/employee` .

So this controller handles requests to:

```
http://localhost:8080/employee
```

Counter variable

```java
private int count = 0;
```

This variable stores how many times the method has been called.

Because controller is **singleton by default**, this variable value is preserved across requests.

API method

Java

```java
@GetMapping
```

This means this method handles HTTP GET request.

Since class path is `/employee`, full path becomes:

```
GET /employee
```

Method declaration

Java

```java
public Map<String, Object> getEmployees()
```

This method returns a map that contains:

- invocation count
- employee list

Spring converts it to JSON automatically.

Increase count

Java

```java
count++;
```

Each time the method is called, increment counter by 1.

Create employee list

Java

```java
List<Employee> employees = Arrays.asList(
        new Employee(1, "Asha", 50000),
        new Employee(2, "Ravi", 60000),
        new Employee(3, "Kiran", 70000)
);
```

Creates a fixed list of employees.

Create response map

```java
Map<String, Object> response = new HashMap<>();
```

Creates a map to return multiple values in one JSON response.

Add count

```java
response.put("invocationCount", count);
```

Adds count into response.

Add employee list

```java
response.put("employees", employees);
```

Adds employee list into response.

Return response

```java
return response;
```

Spring converts this map into JSON and sends it back to client.

---

8) Per-request lifecycle version

If your faculty explicitly asks for **Per-Request lifecycle**, then use this:

```java
```

```java
package com.example.demo.controller;

import com.example.demo.model.Employee;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.context.annotation.RequestScope;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/employee")
@RequestScope
public class EmployeeController {

    private int count = 0;

    @GetMapping
    public Map<String, Object> getEmployees() {
```

What happens here?

Because of:

```java
@RequestScope
```

a new controller object is created for every request.

So:

- first request → count = 1
- second request → again count = 1
- third request → again count = 1

So this does not maintain total invocation count across requests.

---

9) Which one should you write in exam?

If question says:

"display count of number of times method was invoked"

Then write **Singleton lifecycle** because it correctly maintains running count.

You can also mention:

> Spring beans are singleton by default.
> Singleton scope is suitable here because invocation count must persist across multiple requests.
> In request scope, count resets for every request.

That will make your answer stronger.

---

10) Postman testing

Request

- Method: **GET**
- URL:

```
http://localhost:8080/employee
```

Result

Call multiple times and check:

- Singleton version → count increases
- Request scope version → count always 1

---

11) Very short exam answer version

Java

```java
@RestController
@RequestMapping("/employee")
public class EmployeeController {

    private int count = 0;

    @GetMapping
    public Map<String, Object> getEmployees() {
        count++;

        List<Employee> employees = Arrays.asList(
                new Employee(1, "Asha", 50000),
                new Employee(2, "Ravi", 60000),
                new Employee(3, "Kiran", 70000)
        );
```

```
        Map<String, Object> response = new HashMap<>();
        response.put("count", count);
        response.put("employees", employees);

        return response;
```

---

12) Interview-style explanation

- REST endpoint path: `/employee`
- HTTP method: `GET`
- returns employee list
- count is stored in controller variable
- Spring controller is singleton by default
- hence invocation count is preserved across requests

---

13) One important improvement

In real applications, using a plain `int` counter in singleton is not ideal for concurrent requests. Better use:

```java
private AtomicInteger count = new AtomicInteger();
```

and then:

```java
int currentCount = count.incrementAndGet();
```

But for beginner/exam level, `int count++` is usually accepted.

If you want, I can now give you the **full runnable Spring Boot project code with `pom.xml`, main class, model, and controller together**.

yes please

Here is the **full runnable Spring Boot project** for your question.

Project structure

```
employee-demo
 └─ src
    └─ main
       ├─ java
       │  └─ com
       │     └─ example
       │        └─ demo
       │           ├─ DemoApplication.java
       │           ├─ controller
       │           │  └─ EmployeeController.java
       │           └─ model
       │              └─ Employee.java
       └─ resources
          └─ application.properties
```

---

1) `pom.xml`

Use this if you are on **Java 17**.

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.ap

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>employee-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>employee-demo</name>
    <description>Simple Employee REST API</description>

    <properties>
```

2) DemoApplication.java

Path:

`src/main/java/com/example/demo/DemoApplication.java`

Java

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

3) Employee.java

Path:

`src/main/java/com/example/demo/model/Employee.java`

Java

```java
package com.example.demo.model;

public class Employee {

    private int id;
    private String name;
    private double salary;

    public Employee() {
    }

    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
```

```
    public int getId() {
        return id;
    }
```

---

4) EmployeeController.java

Path:

`src/main/java/com/example/demo/controller/EmployeeController.java`

Java

```java
package com.example.demo.controller;

import com.example.demo.model.Employee;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/employee")
public class EmployeeController {

    private int count = 0;

    @GetMapping
    public Map<String, Object> getEmployees() {
        count++;
```

---

5) application.properties

Path:

`src/main/resources/application.properties`

properties

```properties
spring.application.name=employee-demo
server.port=8080
```

What this does

When you run the project and open:

```
http://localhost:8080/employee
```

you will get response like:

JSON

```json
{
  "count": 1,
  "employees": [
    {
      "id": 1,
      "name": "Asha",
      "salary": 50000.0
    },
    {
      "id": 2,
      "name": "Ravi",
      "salary": 60000.0
    },
    {
      "id": 3,
      "name": "Kiran",
      "salary": 70000.0
    }
  ]
}
```

If you call again:

JSON

```json
{
  "count": 2,
  "employees": [
    {
      "id": 1,
      "name": "Asha",
      "salary": 50000.0
    },
    {
      "id": 2,
      "name": "Ravi",
      "salary": 60000.0
    },
    {
      "id": 3,
```

```
      "name": "Kiran",
      "salary": 70000.0
    }
  ]
}
```

How to create this in Eclipse

Step 1

Create a **Spring Starter Project**

- Name: `employee-demo`
- Group: `com.example`
- Artifact: `employee-demo`
- Packaging: `jar`
- Java: `17`
- Dependency: `Spring Web`

Step 2

Create package:

- `com.example.demo.model`
- `com.example.demo.controller`

Step 3

Add the 3 Java files above

Step 4

Run `DemoApplication.java`

Right click → **Run As** → **Spring Boot App**

Step 5

Test in browser or Postman:

```
GET http://localhost:8080/employee
```

Line-by-line quick meaning

`@RestController`

Makes this class a REST API controller.

`@RequestMapping("/employee")`

Base path is `/employee`.

`private int count = 0;`

Stores how many times the API was called.

`@GetMapping`

This method handles GET request.

`count++;`

Increases count every time method is called.

`List<Employee> employees = Arrays.asList(...)`

Creates list of employees.

`Map<String, Object> response = new HashMap<>();`

Creates response object.

`response.put("count", count);`

Adds invocation count.

`response.put("employees", employees);`

Adds employee list.

`return response;`

Returns JSON response.

Per-request version

If your teacher specifically asks for request scope version, change controller to this:

Java

```java
package com.example.demo.controller;

import com.example.demo.model.Employee;
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.context.annotation.RequestScope;

import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/employee")
@RequestScope
public class EmployeeController {

    private int count = 0;

    @GetMapping
```

Difference

- without `@RequestScope` → singleton, count keeps increasing
- with `@RequestScope` → new object per request, count resets

Best exam note

Write this sentence too:

Spring controllers are singleton by default. Therefore, singleton scope is suitable when we need to maintain invocation count across multiple requests.

If you want, next I can give you:

1. the same project with **very simple comments on every line**, or
2. **Postman testing steps with screenshots-style explanation**.

Postman testing steps with screenshots-style explanation.

Perfect — let's do **Postman testing step-by-step with "screenshots-style" guidance** (I'll describe exactly what you should see and click).

🎯 Goal

Test this API:

```
GET http://localhost:8080/employee
```

✅ Step 0: Start your Spring Boot app

In Eclipse:

👉 Right click `DemoApplication.java`
👉 **Run As → Spring Boot App**

You should see in console:

```
Tomcat started on port(s): 8080
```

🚀 Step 1: Open Postman

Open Postman → You will see a screen like:

```
+ New Tab
[ GET ▼ ]  Enter request URL here...
```

🧭 Step 2: Set HTTP Method

Top-left dropdown:

👉 Change from `GET` (default) → keep it as **GET**

(For this API, we are using GET)

🌐 Step 3: Enter URL

In URL bar, type:

```
http://localhost:8080/employee
```

So it should look like:

```
[ GET ▼ ]  http://localhost:8080/employee   [ Send ]
```

▶️ Step 4: Click Send

👉 Click **Send**

📥 Step 5: See Response

You will see output like:

```JSON
{
  "count": 1,
  "employees": [
    {
      "id": 1,
      "name": "Asha",
      "salary": 50000.0
    },
    {
      "id": 2,
      "name": "Ravi",
      "salary": 60000.0
    },
    {
      "id": 3,
      "name": "Kiran",
      "salary": 70000.0
    }
  ]
}
```

🔁 Step 6: Test lifecycle (very important)

Click **Send again**

Now response becomes:

```JSON
```

```
{
  "count": 2,
  "employees": [...]
}
```

👉 This proves:

- your API is working
- **Singleton lifecycle is working**
- count is increasing

🔍 What each part of Postman screen means

Top section

```
[ GET ▼ ]  URL   [ Send ]
```

- GET → method
- URL → API endpoint
- Send → trigger request

Response section

You'll see tabs like:

- **Body** → JSON response (main output)
- **Headers** → response headers
- **Status** → should show:

```
Status: 200 OK
```

⚠️ Common errors + fixes

❌ Error: Could not send request

👉 Fix:

- Make sure Spring Boot is running

❌ 404 Not Found

👉 Fix:

- Check URL spelling:

```
/employee
```

- Check controller:

```java
@RequestMapping("/employee")
```

❌ Connection refused

👉 Fix:

- App not running
- Wrong port

❌ Count not increasing

👉 Fix:

- Check you DID NOT use:

```java
@RequestScope
```

- Singleton should be default

🧪 Bonus: Test using Browser (super easy)

Just open browser:

```
http://localhost:8080/employee
```

Refresh multiple times → count increases

🧠 What you learned

Concept	What you did
API call	GET request
Endpoint	`/employee`
Testing tool	Postman
Response	JSON
Lifecycle	Singleton verified

🔥 Next level (optional)

If you want, next I can show you:

- POST API (create employee)
- PUT API (update employee)
- DELETE API
- Full CRUD testing in Postman

Just tell 👍

2. Create a RESTful Web Service and Test it as per the below requirements:

Sensitivity: Internal & Restricted

a. The restful web service is available on the path "jobs".
b. The web service provides two methods.
c. The first method allows the "job seeker" to upload his/her CV/Profile document.
d. The second method allows the "job administrators" to download the CV/Profiles uploaded by the job seekers.

JAVA + SPRING BOOT

Here is a complete **Java + Spring Boot** solution for requirement 2.

Requirement understood

Create a REST API on path `/jobs` with **2 methods**:

- method 1: job seeker uploads CV/Profile
- method 2: job administrator downloads uploaded CV/Profile

This is basically a **file upload + file download API**.

Project structure

```
jobs-demo
 └─ src
    └─ main
       ├─ java
       │  └─ com
       │     └─ example
       │        └─ jobsdemo
       │           ├─ JobsDemoApplication.java
       │           └─ controller
       │              └─ JobController.java
       └─ resources
          └─ application.properties
```

---

1) `pom.xml`

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.ap

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>jobs-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>jobs-demo</name>
```

```
    <description>Job upload download API</description>

    <properties>
```

---

2) JobsDemoApplication.java

Path:

`src/main/java/com/example/jobsdemo/JobsDemoApplication.java`

Java

```java
package com.example.jobsdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JobsDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(JobsDemoApplication.class, args);
    }
}
```

---

3) JobController.java

Path:

`src/main/java/com/example/jobsdemo/controller/JobController.java`

Java

```java
package com.example.jobsdemo.controller;

import org.springframework.core.io.InputStreamResource;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.*;

@RestController
@RequestMapping("/jobs")
public class JobController {
```

```
    private static final String UPLOAD_DIR = "uploads";

    @PostMapping("/upload")
    public ResponseEntity<String> uploadCV(@RequestParam("file") MultipartFile
        try {
            File dir = new File(UPLOAD_DIR);
```

---

4) `application.properties`

Path:

`src/main/resources/application.properties`

properties

```properties
spring.application.name=jobs-demo
server.port=8080

spring.servlet.multipart.enabled=true
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

What this API does

Upload API

Path:

```
POST /jobs/upload
```

Used by:

- job seeker

Purpose:

- upload CV/Profile file

Download API

Path:

```
GET /jobs/download/{fileName}
```

Used by:

- job administrator

Purpose:

- download uploaded CV/Profile file

Postman testing steps

1) Upload CV/Profile

Method

`POST`

URL

```
http://localhost:8080/jobs/upload
```

In Postman

- choose **Body**
- select **form-data**
- key = `file`
- type = **File**
- choose your resume file

Then click **Send**

Success response

```
File uploaded successfully: resume.pdf
```

---

2) Download CV/Profile

Method

`GET`

URL

Example:

```
http://localhost:8080/jobs/download/resume.pdf
```

Then click **Send**

If file exists, it gets downloaded.

Line-by-line explanation

`@RestController`

```java
@RestController
```

This class is a REST API controller.

It returns HTTP responses.

`@RequestMapping("/jobs")`

```java
@RequestMapping("/jobs")
```

Base path for all methods in this controller is `/jobs`.

So:

- upload path becomes `/jobs/upload`
- download path becomes `/jobs/download/{fileName}`

`UPLOAD_DIR`

```java
private static final String UPLOAD_DIR = "uploads";
```

This is the folder where uploaded files will be stored.

If your project runs, a folder named `uploads` will be created.

Upload method

`@PostMapping("/upload")`

Java

```java
@PostMapping("/upload")
```

This handles HTTP POST request for file upload.

Full path:

```
POST /jobs/upload
```

Method declaration

Java

```java
public ResponseEntity<String> uploadCV(@RequestParam("file") MultipartFile file)
```

Meaning:

- `ResponseEntity<String>` → returns HTTP response with text message
- `@RequestParam("file")` → reads uploaded file from request
- `MultipartFile file` → Spring stores uploaded file here

In Postman, the key name must be exactly:

```
file
```

Create upload folder

Java

```java
File dir = new File(UPLOAD_DIR);
if (!dir.exists()) {
    dir.mkdirs();
}
```

If `uploads` folder does not exist, create it.

Get original file name

```java
String fileName = file.getOriginalFilename();
```

Gets uploaded file name, for example:

```
resume.pdf
```

Create saved file object

```java
File savedFile = new File(dir, fileName);
```

Creates file path like:

```
uploads/resume.pdf
```

Write file to folder

```java
try (FileOutputStream fos = new FileOutputStream(savedFile)) {
    fos.write(file.getBytes());
}
```

This saves uploaded file into local folder.

Success response

```java
return ResponseEntity.ok("File uploaded successfully: " + fileName);
```

Returns:

- status `200 OK`

- success message

Error response

```java
return ResponseEntity.internalServerError()
        .body("File upload failed: " + e.getMessage());
```

If error happens, return:

- status `500`
- failure message

Download method

`@GetMapping("/download/{fileName}")`

```java
@GetMapping("/download/{fileName}")
```

This handles GET request for download.

Example URL:

```
GET /jobs/download/resume.pdf
```

Method declaration

```java
public ResponseEntity<InputStreamResource> downloadCV(@PathVariable String fileNa
```

Meaning:

- gets `fileName` from URL
- returns file as response stream

Create file object

```java
File file = new File(UPLOAD_DIR, fileName);
```

Looks for file inside uploads folder.

Check file exists

```java
if (!file.exists()) {
    return ResponseEntity.notFound().build();
}
```

If file not found, return:

- `404 Not Found`

Create file resource

```java
InputStreamResource resource = new InputStreamResource(new FileInputStream(file))
```

This reads the file for download.

Return downloadable response

```java
return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment;filename=" + file.ge
        .contentType(MediaType.APPLICATION_OCTET_STREAM)
        .contentLength(file.length())
        .body(resource);
```

What it does:

- `200 OK`

- tells browser/Postman this is an attachment
- returns the file content

Exam-style short answer

If you need a shorter exam answer, use this controller:

Java

```java
package com.example.jobsdemo.controller;

import org.springframework.core.io.InputStreamResource;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.*;

@RestController
@RequestMapping("/jobs")
public class JobController {

    private static final String UPLOAD_DIR = "uploads";

    @PostMapping("/upload")
    public ResponseEntity<String> uploadCV(@RequestParam("file") MultipartFile
        try {
            File dir = new File(UPLOAD_DIR);
            if (!dir.exists()) dir.mkdirs();
```

Important note

This solution is good for:

- learning
- exam
- basic demo

In real applications, you should add:

- file type validation
- file size validation
- security/authentication
- store file metadata in database
- avoid unsafe file names

How to test in Postman

Upload

- Method: `POST`
- URL: `http://localhost:8080/jobs/upload`
- Body → `form-data`
- key: `file`
- type: `File`
- choose file
- click **Send**

Download

- Method: `GET`
- URL: `http://localhost:8080/jobs/download/resume.pdf`
- click **Send**

If you want, I can next give you:

1. **Postman screenshots-style testing steps for upload and download**, or
2. **the same code with comments on every line for study/exam writing**.

both 1 and 2 one by one

Great — we'll do this **step-by-step, clean and clear**.

✅ PART 1: Postman Testing (Screenshots-style explanation)

🎯 We will test:

1. Upload CV (POST)
2. Download CV (GET)

🔹 A) Upload CV using Postman

Step 1: Open Postman

You'll see:

```
[ GET ▼ ]  Enter request URL   [ Send ]
```

Step 2: Change method to POST

Click dropdown → select:

```
POST
```

Step 3: Enter URL

```
http://localhost:8080/jobs/upload
```

Now screen looks like:

```
[ POST ▼ ]  http://localhost:8080/jobs/upload   [ Send ]
```

Step 4: Go to Body tab

Click:
👉 **Body**

Then select:
👉 **form-data**

Step 5: Add file field

You will see table like:

KEY	VALUE	TYPE

Fill like this:

KEY	VALUE	TYPE
file	(choose your file)	File

👉 Important:

- KEY must be exactly: **file**
- TYPE must be **File**

Step 6: Choose file

Click **Select Files**
Choose your resume (example: `resume.pdf`)

Step 7: Click Send

👉 Click **Send**

✅ Output

You will see:

```
Status: 200 OK
```

Body:

```
File uploaded successfully: resume.pdf
```

👉 Also check your project folder — a new folder:

```
uploads/
   resume.pdf
```

🔹 B) Download CV using Postman

Step 1: Change method to GET

```
GET
```

Step 2: Enter URL

Example:

```
http://localhost:8080/jobs/download/resume.pdf
```

Step 3: Click Send

✅ Output

- File will be downloaded automatically
  OR
- You will see binary data in Postman

❌ If file not found

You will see:

```
Status: 404 Not Found
```

⚠️ Common mistakes

Mistake	Fix
Using JSON instead of form-data	Use form-data
Wrong key name	Must be `file`
Server not running	Start Spring Boot
Wrong file name in download	Match exact name

🧠 What you learned

Action	API
Upload file	POST `/jobs/upload`
Download file	GET `/jobs/download/{fileName}`

✅ PART 2: Code with LINE-BY-LINE explanation

Now I'll explain your controller like a beginner.

Full Controller

Java

```java
@RestController
@RequestMapping("/jobs")
public class JobController {

    private static final String UPLOAD_DIR = "uploads";
```

🔹 Line 1

Java

```java
@RestController
```

👉 Meaning:

- This class is a REST API
- returns JSON / response directly

🔹 Line 2

Java

```java
@RequestMapping("/jobs")
```

👉 Meaning:

- base path is `/jobs`

So all APIs start with:

```
/jobs
```

🔹 Line 3

```java
private static final String UPLOAD_DIR = "uploads";
```

👉 Meaning:

- folder name where files will be stored

🔸 Upload Method

```java
@PostMapping("/upload")
```

👉 Meaning:

- this method handles **POST request**
- URL:

```
POST /jobs/upload
```

```java
public ResponseEntity<String> uploadCV(@RequestParam("file") MultipartFile file)
```

👉 Meaning:

- takes uploaded file from request
- `file` must match Postman key

Java

```java
File dir = new File(UPLOAD_DIR);
```

👉 creates folder object

Java

```java
if (!dir.exists()) {
    dir.mkdirs();
}
```

👉 if folder doesn't exist → create it

Java

```java
String fileName = file.getOriginalFilename();
```

👉 gets file name (example: resume.pdf)

Java

```java
File savedFile = new File(dir, fileName);
```

👉 creates path:

```
uploads/resume.pdf
```

Java

```java
try (FileOutputStream fos = new FileOutputStream(savedFile)) {
    fos.write(file.getBytes());
}
```

👉 saves file to disk

Java

```java
return ResponseEntity.ok("File uploaded successfully: " + fileName);
```

👉 returns:

- status 200
- success message

Java

```java
catch (Exception e)
```

👉 handles errors

Java

```java
return ResponseEntity.internalServerError()
```

👉 returns 500 error if something fails

🔹 Download Method

Java

```java
@GetMapping("/download/{fileName}")
```

👉 Meaning:

- GET API for download

Example:

```
/jobs/download/resume.pdf
```

Java

```java
public ResponseEntity<InputStreamResource> downloadCV(@PathVariable String fileNa
```

👉 gets file name from URL

Java

```java
File file = new File(UPLOAD_DIR, fileName);
```

👉 looks for file in:

```
uploads/
```

Java

```java
if (!file.exists()) {
    return ResponseEntity.notFound().build();
}
```

👉 if file not found → return 404

Java

```java
InputStreamResource resource = new InputStreamResource(new FileInputStream(file))
```

👉 reads file content

Java

```java
return ResponseEntity.ok()
```

👉 return success response

Java

```java
.header(HttpHeaders.CONTENT_DISPOSITION, "attachment;filename=" + file.getName())
```

👉 tells browser/Postman:
"download this file"

```java
.contentType(MediaType.APPLICATION_OCTET_STREAM)
```

👉 generic file type

```java
.contentLength(file.length())
```

👉 file size

```java
.body(resource);
```

👉 send file as response

🔥 Final understanding

Feature	Explanation
Upload	user sends file
Download	admin gets file
MultipartFile	represents uploaded file
FileOutputStream	saves file
InputStreamResource	sends file
ResponseEntity	controls response

👍 If you want next

I can help you with:

- add **database (store file info)**
- add **security (admin vs user roles)**
- or convert this into **real project-level structure (service layer, DTO, etc.)**

Just tell 👍

2/2

3. Create a RESTful Web Services and Test it as per the below requirements:
   a. The restful web services is available on the path "movies".
   b. The web service helps in managing the details of movies like movieID, movieName, leadActor, leadActress, movieDirector and totalCollection.
   c. The restful web service should support the below services pertaining to movies details:
      i. "GET" method that returns all the movie detail
      ii. "GET" method that returns the movie detail based on the "movieID" entered
      iii. "POST" method to add new movie entries.
      iv. "PUT" method to update existing movie entries.
      v. "DELETE" method at present is NOT SUPPORTED.

   d. Please implement it **using "RESTEasy" JAX-RS reference Implementation (RI) ONLY (**Important)**
   e. You can use any database/or collection structures of your choice.

JAVA + SPRING BOOT

There is one important thing first:

Your screenshot says:

"Please implement it using RESTEasy JAX-RS reference implementation (RI) ONLY"

That is **not the normal Spring Boot style**.
So you have 2 possible ways:

- **Exam-correct answer:** use **JAX-RS / RESTEasy style**
- **Beginner-friendly learning answer:** use **Spring Boot style**

Since you wrote **"JAVA + SPRING BOOT"**, I'll give you the solution in **Spring Boot**, and I'll also tell you the small difference from RESTEasy so you do not get confused.

What you need to build

Path: `/movies`

Movie fields:

- `movieID`
- `movieName`
- `leadActor`
- `leadActress`
- `movieDirector`
- `totalCollection`

Supported APIs:

- `GET /movies` → get all movies
- `GET /movies/{movieID}` → get one movie by id
- `POST /movies` → add movie
- `PUT /movies/{movieID}` → update movie
- `DELETE` → **not supported**

We will use **collection structure** (`Map`) instead of database, which matches your requirement.

Project structure

```
movies-demo
 └─ src
    └─ main
       ├─ java
       │  └─ com
       │     └─ example
       │        └─ moviesdemo
       │           ├─ MoviesDemoApplication.java
       │           ├─ controller
       │           │  └─ MovieController.java
       │           └─ model
       │              └─ Movie.java
       └─ resources
          └─ application.properties
```

---

1) `pom.xml`

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.ap

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>movies-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>movies-demo</name>
    <description>Movie REST API</description>

    <properties>
        <java.version>17</java.version>
```

---

2) MoviesDemoApplication.java

Path:

`src/main/java/com/example/moviesdemo/MoviesDemoApplication.java`

Java

```java
package com.example.moviesdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MoviesDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(MoviesDemoApplication.class, args);
    }
}
```

---

3) Movie.java

Path:

`src/main/java/com/example/moviesdemo/model/Movie.java`

Java

```java
package com.example.moviesdemo.model;

public class Movie {

    private String movieID;
    private String movieName;
    private String leadActor;
    private String leadActress;
    private String movieDirector;
    private double totalCollection;

    public Movie() {
    }

    public Movie(String movieID, String movieName, String leadActor, String lea
                 String movieDirector, double totalCollection) {
        this.movieID = movieID;
        this.movieName = movieName;
        this.leadActor = leadActor;
        this.leadActress = leadActress;
        this.movieDirector = movieDirector;
        this.totalCollection = totalCollection;
```

---

4) MovieController.java

Path:

`src/main/java/com/example/moviesdemo/controller/MovieController.java`

Java

```java
package com.example.moviesdemo.controller;

import com.example.moviesdemo.model.Movie;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import jakarta.annotation.PostConstruct;
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

@RestController
```

```
@RequestMapping("/movies")
public class MovieController {

    private final Map<String, Movie> movies = new HashMap<>();

    @PostConstruct
    public void init() {
        movies.put("M101", new Movie("M101", "Inception", "Leonardo DiCaprio"
```

---

5) `application.properties`

Path:

`src/main/resources/application.properties`

properties

```properties
spring.application.name=movies-demo
server.port=8080
```

What each API does

1. Get all movies

http

```http
GET /movies
```

Returns all movie details.

---

2. Get movie by ID

http

```http
GET /movies/{movieID}
```

Example:

http

```http
GET /movies/M101
```

Returns only one movie.

3. Add movie

```http
POST /movies
```

Adds a new movie.

---

4. Update movie

```http
PUT /movies/{movieID}
```

Example:

```http
PUT /movies/M101
```

Updates an existing movie.

---

5. Delete movie

```http
DELETE /movies/{movieID}
```

Returns:

- `405 Method Not Allowed`

because the requirement says delete is not supported.

Postman testing steps

A) GET all movies

Method: `GET`

URL:

```
http://localhost:8080/movies
```

Click **Send**

Response

JSON

```
[
  {
    "movieID": "M101",
    "movieName": "Inception",
    "leadActor": "Leonardo DiCaprio",
    "leadActress": "Elliot Page",
    "movieDirector": "Christopher Nolan",
    "totalCollection": 950.0
  },
  {
    "movieID": "M102",
    "movieName": "Titanic",
    "leadActor": "Leonardo DiCaprio",
    "leadActress": "Kate Winslet",
    "movieDirector": "James Cameron",
    "totalCollection": 2200.0
  }
]
```

B) GET movie by ID

Method: `GET`

URL:

```
http://localhost:8080/movies/M101
```

Response

JSON

```
{
  "movieID": "M101",
  "movieName": "Inception",
  "leadActor": "Leonardo DiCaprio",
  "leadActress": "Elliot Page",
  "movieDirector": "Christopher Nolan",
```

```
  "totalCollection": 950.0
}
```

C) POST add movie

Method: `POST`

URL:

```
http://localhost:8080/movies
```

Body → `raw` → `JSON`

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 2900.0
}
```

Response

```
Movie added successfully
```

Status:

```
201 Created
```

D) PUT update movie

Method: `PUT`

URL:

```
http://localhost:8080/movies/M103
```

Body → `raw` → `JSON`

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar Updated",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 3000.0
}
```

Response

```
Movie updated successfully
```

Status:

```
200 OK
```

E) DELETE not supported

Method: `DELETE`

URL:

```
http://localhost:8080/movies/M103
```

Response

```
DELETE method is not supported
```

Status:

```
405 Method Not Allowed
```

Line-by-line explanation of controller

`@RestController`

‹/› Java

```java
@RestController
```

Tells Spring this class is a REST API controller.

`@RequestMapping("/movies")`

```java
@RequestMapping("/movies")
```

Base path for all APIs is `/movies` .

`private final Map<String, Movie> movies = new HashMap<>();`

Stores movie data in memory using a `Map` .

- key = `movieID`
- value = `Movie object`

`@PostConstruct`

```java
@PostConstruct
public void init()
```

Runs automatically once when the app starts.

Used here to add sample movie records.

`@GetMapping`

```java
@GetMapping
public Collection<Movie> getAllMovies()
```

Handles:

```http
GET /movies
```

Returns all movies.

`@GetMapping("/{movieID}")`

```java
@GetMapping("/{movieID}")
public ResponseEntity<Movie> getMovieById(@PathVariable String movieID)
```

Handles:

```http
GET /movies/M101
```

Gets the movie ID from URL.

`movies.get(movieID)`

Searches movie by ID from map.

`@PostMapping`

```java
@PostMapping
public ResponseEntity<String> addMovie(@RequestBody Movie movie)
```

Handles:

```http
POST /movies
```

Gets movie data from JSON request body.

`movies.containsKey(movie.getMovieID())`

Checks whether movie ID already exists.

`movies.put(movie.getMovieID(), movie)`

Adds movie to map.

`HttpStatus.CREATED`

Returns `201 Created` after successful add.

`@PutMapping("/{movieID}")`

Java

```java
@PutMapping("/{movieID}")
public ResponseEntity<String> updateMovie(@PathVariable String movieID, @RequestB
```

Handles update request.

`movie.setMovieID(movieID)`

Ensures the ID from URL is used.

`movies.put(movieID, movie)`

Updates the existing movie.

`@DeleteMapping("/{movieID}")`

We intentionally return method not allowed.

That matches your requirement:
DELETE is NOT SUPPORTED

Important note about your screenshot

Your screenshot says to use **RESTEasy JAX-RS RI only**.
This code is **Spring Boot REST**, not RESTEasy JAX-RS.

So if this is for strict exam evaluation, your faculty may expect annotations like:

- `@Path`
- `@GET`
- `@POST`
- `@PUT`
- `@Consumes`
- `@Produces`

instead of Spring Boot annotations like:

- `@RestController`
- `@GetMapping`
- `@PostMapping`
- `@PutMapping`

Mapping between both styles

Spring Boot	JAX-RS / RESTEasy
`@RestController`	`@Path`
`@GetMapping`	`@GET`
`@PostMapping`	`@POST`
`@PutMapping`	`@PUT`
`@RequestBody`	method parameter with `@Consumes`
`@PathVariable`	`@PathParam`

So if you need the **strict RESTEasy version**, say "**give me the JAX-RS / RESTEasy version for the same requirement**" and I'll write that next.

give me the JAX-RS / RESTEasy version for the same requirement

Here is the **JAX-RS / RESTEasy version** for the same requirement.

This matches the style your question is asking for:

- `@Path`
- `@GET`
- `@POST`
- `@PUT`
- `@Produces`
- `@Consumes`

Not Spring MVC annotations.

What this service does

Base path: `/movies`

Supports:

- `GET /movies` → get all movies
- `GET /movies/{movieID}` → get movie by ID
- `POST /movies` → add new movie
- `PUT /movies/{movieID}` → update movie
- `DELETE` → not supported

We will use a **Map collection** as storage.

---

1) `Movie.java`

Java

```java
package com.example.model;

import jakarta.xml.bind.annotation.XmlRootElement;

@XmlRootElement
public class Movie {

    private String movieID;
    private String movieName;
    private String leadActor;
    private String leadActress;
    private String movieDirector;
    private double totalCollection;
```

```
    public Movie() {
    }

    public Movie(String movieID, String movieName, String leadActor, String lea
                 String movieDirector, double totalCollection) {
        this.movieID = movieID;
        this.movieName = movieName;
```

---

2) `MovieResource.java`

Java

```
package com.example.resource;

import com.example.model.Movie;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

@Path("/movies")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class MovieResource {

    private static Map<String, Movie> movies = new HashMap<>();

    static {
        movies.put("M101", new Movie("M101", "Inception", "Leonardo DiCaprio",
        movies.put("M102", new Movie("M102", "Titanic", "Leonardo DiCaprio", "K
    }
```

---

3) If your server needs JAX-RS application config

Create `MyApplication.java`

Java

```
package com.example.config;

import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("/api")
```

```
public class MyApplication extends Application {
}
```

Final path becomes

If you use:

```java
@ApplicationPath("/api")
```

and resource has:

```java
@Path("/movies")
```

then full URL becomes:

```
http://localhost:8080/api/movies
```

If your server setup already maps JAX-RS directly, then it may just be:

```
http://localhost:8080/movies
```

Postman URLs

1) Get all movies

```
GET http://localhost:8080/api/movies
```

2) Get movie by ID

```
GET http://localhost:8080/api/movies/M101
```

3) Add new movie

```
POST http://localhost:8080/api/movies
```

Body:

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 2900.0
}
```

4) Update movie

```
PUT http://localhost:8080/api/movies/M103
```

Body:

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar Updated",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 3000.0
}
```

Line-by-line explanation

`@Path("/movies")`

This sets the base URL path for the resource.

All methods inside this class work under `/movies`.

`@Produces(MediaType.APPLICATION_JSON)`

This tells RESTEasy:
return output in **JSON format**.

`@Consumes(MediaType.APPLICATION_JSON)`

This tells RESTEasy:
accept input in **JSON format**.

```
private static Map<String, Movie> movies = new HashMap<>();
```

This stores movie data in memory.

- key = movie ID
- value = movie object

`static` means shared for all requests.

```
static { ... }
```

This block adds sample movies when the class loads.

```
@GET
```

This marks the method as HTTP GET.

```
public Collection<Movie> getAllMovies()
```

Returns all movie values from the map.

```
@Path("/{movieID}")
```

This defines a dynamic path parameter.

Example:

```
/movies/M101
```

```
@PathParam("movieID") String movieID
```

Reads the value from URL path.

So if URL is:

```
/movies/M101
```

then:

```java
movieID = "M101"
```

`Response.status(...).entity(...).build()`

Creates full HTTP response with:

- status code
- body message/data

`@POST`

This marks the method for HTTP POST.

Used to add new movie.

`public Response addMovie(Movie movie)`

RESTEasy automatically converts JSON body into `Movie` object.

`@PUT`

This marks the method for HTTP PUT.

Used to update an existing movie.

`movie.setMovieID(movieID)`

Ensures movie ID in URL is used as the correct ID.

DELETE not supported

Your requirement says DELETE is not supported.

So we simply do not add a `@DELETE` method.

That is usually enough.

If you want to explicitly show it, you can add this:

Java

```java
@DELETE
@Path("/{movieID}")
public Response deleteMovie(@PathParam("movieID") String movieID) {
    return Response.status(Response.Status.METHOD_NOT_ALLOWED)
            .entity("DELETE method is not supported")
            .build();
}
```

JAX-RS vs Spring Boot annotation mapping

JAX-RS / RESTEasy	Spring Boot
`@Path`	`@RequestMapping`
`@GET`	`@GetMapping`
`@POST`	`@PostMapping`
`@PUT`	`@PutMapping`
`@PathParam`	`@PathVariable`
`Response`	`ResponseEntity`
`@Produces`	automatic JSON or `produces`
`@Consumes`	automatic JSON or `consumes`

Exam-ready short version

Java

```java
@Path("/movies")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
```

```
public class MovieResource {

    private static Map<String, Movie> movies = new HashMap<>();

    static {
        movies.put("M101", new Movie("M101", "Inception", "Leonardo DiCaprio",
    }

    @GET
    public Collection<Movie> getAllMovies() {
        return movies.values();
    }

    @GET
    @Path("/{movieID}")
    public Response getMovieById(@PathParam("movieID") String movieID) {
        Movie movie = movies.get(movieID);
        if (movie != null)
            return Response.status(200).entity(movie).build();
```

Important note

This is **JAX-RS / RESTEasy style**, but it is **not Spring Boot MVC style**.

So if your faculty says **RESTEasy only**, this is the right pattern to write.

If you want next, I can give you:

1. **full Maven `pom.xml` for RESTEasy project**, or
2. **Postman testing steps for this JAX-RS version**.

full Maven pom.xml for RESTEasy project then next Postman testing steps for this JAX-RS version

Below is a **full Maven `pom.xml` for a RESTEasy JAX-RS project** first, then the **Postman testing steps** for that version.

1) Full Maven `pom.xml` for RESTEasy project

This version is a **simple WAR project** for RESTEasy running on a servlet container like Tomcat.

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>resteasy-movies-api</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>war</packaging>

    <name>resteasy-movies-api</name>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <resteasy.version>6.2.8.Final</resteasy.version>
        <jakarta.ws.rs.version>3.1.0</jakarta.ws.rs.version>
```

---

2) Supporting files you usually need

For RESTEasy, `pom.xml` alone is not enough. You usually also need:

`MyApplication.java`

Java

```java
package com.example.config;

import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("/api")
public class MyApplication extends Application {
}
```

`Movie.java`

Java

```java
package com.example.model;
```

```
import jakarta.xml.bind.annotation.XmlRootElement;

@XmlRootElement
public class Movie {

    private String movieID;
    private String movieName;
    private String leadActor;
    private String leadActress;
    private String movieDirector;
    private double totalCollection;

    public Movie() {
    }

    public Movie(String movieID, String movieName, String leadActor, String l
                 String movieDirector, double totalCollection) {
        this.movieID = movieID;
        this.movieName = movieName;
```

MovieResource.java

Java

```
package com.example.resource;

import com.example.model.Movie;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

@Path("/movies")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class MovieResource {

    private static Map<String, Movie> movies = new HashMap<>();

    static {
        movies.put("M101", new Movie("M101", "Inception", "Leonardo DiCaprio",
        movies.put("M102", new Movie("M102", "Titanic", "Leonardo DiCaprio", "K
    }
```

---

3) Project structure

```
resteasy-movies-api
 └─ src
    └─ main
       ├─ java
       │  └─ com
       │     └─ example
       │        ├─ config
       │        │  └─ MyApplication.java
       │        ├─ model
       │        │  └─ Movie.java
       │        └─ resource
       │           └─ MovieResource.java
       └─ webapp
          └─ WEB-INF
```

---

4) Postman testing steps for this JAX-RS version

Assuming:

- app is running on `localhost:8080`
- context path is `resteasy-movies-api`
- `@ApplicationPath("/api")` is used

Then the full base URL becomes:

```
http://localhost:8080/resteasy-movies-api/api/movies
```

If your server deploys with a different project name, replace `resteasy-movies-api` with your deployed app name.

A) Test GET all movies

Step 1

Open Postman.

Step 2

Select method:

```
GET
```

Step 3

Enter URL:

```
http://localhost:8080/resteasy-movies-api/api/movies
```

Step 4

Click **Send**

Expected response

JSON

```
[
  {
    "movieID": "M101",
    "movieName": "Inception",
    "leadActor": "Leonardo DiCaprio",
    "leadActress": "Elliot Page",
    "movieDirector": "Christopher Nolan",
    "totalCollection": 950.0
  },
  {
    "movieID": "M102",
    "movieName": "Titanic",
    "leadActor": "Leonardo DiCaprio",
    "leadActress": "Kate Winslet",
    "movieDirector": "James Cameron",
    "totalCollection": 2200.0
  }
]
```

Status should be:

```
200 OK
```

B) Test GET movie by ID

Method

```
GET
```

URL

```
http://localhost:8080/resteasy-movies-api/api/movies/M101
```

Click

Send

Expected response

JSON

```
{
  "movieID": "M101",
  "movieName": "Inception",
  "leadActor": "Leonardo DiCaprio",
  "leadActress": "Elliot Page",
  "movieDirector": "Christopher Nolan",
  "totalCollection": 950.0
}
```

If movie not found

Try:

```
http://localhost:8080/resteasy-movies-api/api/movies/M999
```

Expected:

```
Movie not found
```

Status:

```
404 Not Found
```

C) Test POST add new movie

Method

```
POST
```

URL

```
http://localhost:8080/resteasy-movies-api/api/movies
```

Body setup

Go to:

- **Body**
- choose **raw**
- select **JSON**

Paste this JSON

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 2900.0
}
```

Click

Send

Expected response

```
Movie added successfully
```

Status:

```
201 Created
```

Duplicate ID test

Send the same POST again.

Expected:

```
Movie ID already exists
```

Status:

```
409 Conflict
```

D) Test PUT update movie

Method

```
PUT
```

URL

```
http://localhost:8080/resteasy-movies-api/api/movies/M103
```

Body

raw → JSON

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar Updated",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 3000.0
}
```

Click

Send

Expected response

```
Movie updated successfully
```

Status:

```
200 OK
```

Verify update

Now send:

```
GET http://localhost:8080/resteasy-movies-api/api/movies/M103
```

You should see updated data.

E) Test DELETE not supported

Method

```
DELETE
```

URL

```
http://localhost:8080/resteasy-movies-api/api/movies/M103
```

Click

Send

Expected response

```
DELETE method is not supported
```

Status:

```
405 Method Not Allowed
```

---

5) Screenshots-style quick guide

For GET all

Postman top bar should look like:

```
[ GET ▼ ]  http://localhost:8080/resteasy-movies-api/api/movies   [ Send ]
```

For POST

Top bar:

```
[ POST ▼ ]  http://localhost:8080/resteasy-movies-api/api/movies   [ Send ]
```

Then body:

- Body

- raw
- JSON

Paste JSON and click Send.

For PUT

Top bar:

```
[ PUT ▼ ]  http://localhost:8080/resteasy-movies-api/api/movies/M103   [ Se
```

Body:

- raw
- JSON

Paste updated JSON.

For DELETE

Top bar:

```
[ DELETE ▼ ]  http://localhost:8080/resteasy-movies-api/api/movies/M103   [
```

---

6) Common mistakes

1. Wrong URL

If app name is different, update this part:

```
/resteasy-movies-api/
```

2. Forgot `/api`

Because `MyApplication.java` has:

```java
@ApplicationPath("/api")
```

3. POST/PUT body not set to JSON

Use:

- Body
- raw
- JSON

4. Server not running

Make sure Tomcat/server is started.

5. 404 on every request

Usually means:

- wrong context path
- wrong application path
- project not deployed correctly

---

7) Very important note

This is a **RESTEasy JAX-RS WAR-style project**, not Spring Boot.
So deployment and URL style are different from Spring Boot.

If you want, next I can give you a **full** `web.xml` **version too**, in case your Eclipse/Tomcat setup expects servlet configuration instead of only `@ApplicationPath`.

4. Please implement the previous requirement(Assignment-3) again, but this time ONLY with Spring REST/Spring MVC5 Reference Implementation (RI) ONLY (**Important), with the question 3 answer this 4th one

Yes — for **Question 4**, you should implement the **same Assignment-3 requirement**, but this time using **Spring REST / Spring MVC5 only**.

So the difference is:

- **Question 3** → JAX-RS / RESTEasy
- **Question 4** → Spring REST / Spring MVC5

The functionality stays the same:

- path: `/movies`
- manage movie details
- support:
  - `GET` all movies
  - `GET` by `movieID`
  - `POST` add movie
  - `PUT` update movie
  - `DELETE` not supported

Question 4 Answer — Spring REST / Spring MVC5 Version

Project structure

```
movies-spring-demo
 └─ src
    └─ main
       ├─ java
       │  └─ com
       │     └─ example
       │        └─ moviesdemo
       │           ├─ MoviesDemoApplication.java
       │           ├─ controller
       │           │  └─ MovieController.java
       │           └─ model
       │              └─ Movie.java
       └─ resources
          └─ application.properties
```

---

1) `pom.xml`

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.ap

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>movies-spring-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>movies-spring-demo</name>
    <description>Movie API using Spring REST / Spring MVC</description>

    <properties>
```

---

2) MoviesDemoApplication.java

Java

```java
package com.example.moviesdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MoviesDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(MoviesDemoApplication.class, args);
    }
}
```

---

3) Movie.java

Java

```java
package com.example.moviesdemo.model;

public class Movie {

    private String movieID;
    private String movieName;
    private String leadActor;
    private String leadActress;
    private String movieDirector;
    private double totalCollection;
```

```
    public Movie() {
    }

    public Movie(String movieID, String movieName, String leadActor, String l
                 String movieDirector, double totalCollection) {
        this.movieID = movieID;
        this.movieName = movieName;
        this.leadActor = leadActor;
        this.leadActress = leadActress;
        this.movieDirector = movieDirector;
```

4) MovieController.java

Java

```
package com.example.moviesdemo.controller;

import com.example.moviesdemo.model.Movie;
import jakarta.annotation.PostConstruct;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/movies")
public class MovieController {

    private final Map<String, Movie> movies = new HashMap<>();

    @PostConstruct
    public void init() {
        movies.put("M101", new Movie("M101", "Inception", "Leonardo DiCaprio",
        movies.put("M102", new Movie("M102", "Titanic", "Leonardo DiCaprio", "K
```

5) application.properties

properties

```
spring.application.name=movies-spring-demo
server.port=8080
```

How this satisfies Question 4

Requirement mapping

a) Path should be `movies`

Done here:

```java
@RequestMapping("/movies")
```

---

b) Manage movie details:

- movieID
- movieName
- leadActor
- leadActress
- movieDirector
- totalCollection

Done in `Movie.java`

---

c) Required services

i) GET all movie details

Done here:

```java
@GetMapping
public Collection<Movie> getAllMovies()
```

---

ii) GET movie detail by movieID

Done here:

```java
@GetMapping("/{movieID}")
public ResponseEntity<?> getMovieById(@PathVariable String movieID)
```

iii) POST add new movie

Done here:

```java
@PostMapping
public ResponseEntity<String> addMovie(@RequestBody Movie movie)
```

---

iv) PUT update existing movie

Done here:

```java
@PutMapping("/{movieID}")
public ResponseEntity<String> updateMovie(@PathVariable String movieID, @RequestB
```

---

v) DELETE not supported

Done here:

```java
@DeleteMapping("/{movieID}")
public ResponseEntity<String> deleteMovie(@PathVariable String movieID) {
    return new ResponseEntity<>("DELETE method is not supported", HttpStatus.METH
}
```

---

d) Use Spring REST / Spring MVC5 only

This answer uses Spring annotations only:

- `@RestController`
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@PutMapping`
- `@DeleteMapping`

So this is the correct **Spring REST / Spring MVC** style.

e) Can use database or collection

We used:

```java
Map<String, Movie> movies = new HashMap<>();
```

So we used a **collection structure**, which is allowed.

Postman testing

1) GET all movies

Method:

```
GET
```

URL:

```
http://localhost:8080/movies
```

Response:

```json
[
  {
    "movieID": "M101",
    "movieName": "Inception",
    "leadActor": "Leonardo DiCaprio",
    "leadActress": "Elliot Page",
    "movieDirector": "Christopher Nolan",
    "totalCollection": 950.0
  },
  {
    "movieID": "M102",
    "movieName": "Titanic",
    "leadActor": "Leonardo DiCaprio",
    "leadActress": "Kate Winslet",
    "movieDirector": "James Cameron",
    "totalCollection": 2200.0
  }
]
```

2) GET by movieID

Method:

```
GET
```

URL:

```
http://localhost:8080/movies/M101
```

---

3) POST add movie

Method:

```
POST
```

URL:

```
http://localhost:8080/movies
```

Body → raw → JSON

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 2900.0
}
```

---

4) PUT update movie

Method:

```
PUT
```

URL:

```
http://localhost:8080/movies/M103
```

Body → raw → JSON

JSON

```json
{
  "movieID": "M103",
  "movieName": "Avatar Updated",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 3000.0
}
```

---

5) DELETE not supported

Method:

```
DELETE
```

URL:

```
http://localhost:8080/movies/M103
```

Response:

```
DELETE method is not supported
```

Status:

```
405 Method Not Allowed
```

Line-by-line simple explanation

`@RestController`

Makes this class a REST controller.
Returns JSON/text directly.

`@RequestMapping("/movies")`

Sets base path to `/movies` .

`private final Map<String, Movie> movies = new HashMap<>();`

Stores movie data in memory.

`@PostConstruct`

Runs once when app starts.
Used to add sample movie data.

`@GetMapping`

Handles `GET /movies`

`@GetMapping("/{movieID}")`

Handles `GET /movies/{movieID}`

`@PathVariable String movieID`

Gets movie ID from URL.

`@PostMapping`

Handles `POST /movies`

`@RequestBody Movie movie`

Reads JSON request body and converts it into Java object.

`@PutMapping("/{movieID}")`

Handles update request.

`@DeleteMapping("/{movieID}")`

Handles delete request, but here we return not supported.

Important difference between Question 3 and Question 4

Question 3

Use **RESTEasy / JAX-RS**

Example annotations:

- `@Path`
- `@GET`
- `@POST`
- `@PUT`

Question 4

Use **Spring REST / Spring MVC**

Example annotations:

- `@RestController`
- `@GetMapping`
- `@PostMapping`
- `@PutMapping`

So both questions do the same work, but with **different frameworks**.

Short exam-ready answer

You can write this as the final answer for Question 4:

Java

```java
@RestController
@RequestMapping("/movies")
public class MovieController {
```

```
    private Map<String, Movie> movies = new HashMap<>();

    @GetMapping
    public Collection<Movie> getAllMovies() {
        return movies.values();
    }

    @GetMapping("/{movieID}")
    public ResponseEntity<?> getMovieById(@PathVariable String movieID) {
        Movie movie = movies.get(movieID);
        if (movie != null)
            return new ResponseEntity<>(movie, HttpStatus.OK);
        else
            return new ResponseEntity<>("Movie not found", HttpStatus.NOT_FOUND
    }

    @PostMapping
```

If you want, I can next give you the **same Question 4 answer with comments on every single line**, so you can study it easily.

5. Please add Basic Authentication to the solution of Assignment-3/4 above to ensure the below:
a. “GET” methods need not have any authentication
b. “POST” and “PUT” methods will be guarded through basic authentication.
c. You can use the hardcoded user ID “admin” and password as “wipro@123” for the same.

Great — now we’ll upgrade your **Spring REST (Assignment 4)** solution by adding **Basic Authentication** exactly as required.

🎯 Requirement

- ✅ `GET` → **NO authentication**
- 🔐 `POST` & `PUT` → **require Basic Auth**
- 👤 Username: `admin`
- 🔑 Password: `wipro@123`

✅ How we implement this in Spring Boot

We'll use **Spring Security** and configure:

- allow GET requests without login
- protect POST & PUT requests
- hardcode username/password

---

1) Add dependency in `pom.xml`

Add this inside `<dependencies>` :

XML

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

---

2) Create Security Config Class

📁 Path:

`com.example.moviesdemo.config.SecurityConfig.java`

Java

```java
package com.example.moviesdemo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.password.NoOpPasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception

        http
```

```
            .csrf().disable() // disable CSRF for simplicity
```

---

3) No change needed in your Controller ✅

Your existing controller (Assignment 4) remains SAME.

Security is handled separately.

🔐 How this works

Request	Auth Required?
GET `/movies`	❌ No
GET `/movies/{id}`	❌ No
POST `/movies`	✅ Yes
PUT `/movies/{id}`	✅ Yes
DELETE	❌ allowed but returns 405

🧪 Postman Testing (very important)

🔹 1) Test GET (No Authentication)

Request

```
GET http://localhost:8080/movies
```

👉 Click **Send**

Result

✅ Works WITHOUT login

🔹 2) Test POST WITHOUT Authentication

Request

```
POST http://localhost:8080/movies
```

Body (JSON):

```JSON
{
  "movieID": "M200",
  "movieName": "Test Movie",
  "leadActor": "Actor",
  "leadActress": "Actress",
  "movieDirector": "Director",
  "totalCollection": 1000
}
```

👉 Click **Send**

Result

❌ You get:

```
401 Unauthorized
```

🔹 3) Test POST WITH Authentication

Step 1

Go to **Authorization tab**

Step 2

Select:

```
Type: Basic Auth
```

Step 3

Enter:

```
Username: admin
Password: wipro@123
```

Step 4

Click **Send**

Result

✅ Success:

```
201 Created
Movie added successfully
```

🔹 4) Test PUT WITH Authentication

Method

```
PUT
```

URL

```
http://localhost:8080/movies/M200
```

Authorization

Same:

```
admin / wipro@123
```

Body

‹/› JSON

```
{
  "movieID": "M200",
  "movieName": "Updated Movie",
  "leadActor": "Actor",
  "leadActress": "Actress",
  "movieDirector": "Director",
  "totalCollection": 2000
}
```

👉 Click **Send**

Result

```
200 OK
Movie updated successfully
```

🧠 Line-by-line explanation

`@Configuration`

Marks this class as configuration class.

`SecurityFilterChain`

Controls all HTTP security rules.

`.csrf().disable()`

Disables CSRF protection (needed for Postman testing).

`.requestMatchers(HttpMethod.GET, "/movies/**").permitAll()`

Allows all GET requests without login.

`.requestMatchers(HttpMethod.POST, "/movies/**").authenticated()`

POST requires authentication.

`.requestMatchers(HttpMethod.PUT, "/movies/**").authenticated()`

PUT requires authentication.

`.httpBasic()`

Enables Basic Authentication.

`UserDetailsService`

Stores user credentials in memory.

`User.withUsername("admin")`

Creates user with username `admin`.

`.password("wipro@123")`

Sets password.

`NoOpPasswordEncoder`

Disables encryption (plain text password for learning/demo).

⚠️ Important note

In real applications:

- NEVER use `NoOpPasswordEncoder`
- NEVER store plain text passwords
- Use **BCryptPasswordEncoder**

✅ Final summary (exam-ready)

- Spring Security added using `spring-boot-starter-security`
- Basic Authentication enabled using `.httpBasic()`
- GET methods allowed without authentication
- POST and PUT secured using `.authenticated()`
- Hardcoded user:
  - username: `admin`
  - password: `wipro@123`

If you want next, I can:

- show **same security in JAX-RS / RESTEasy version**

- or explain **Basic Auth vs JWT (interview question)**

6. Enhance the “Basic Authentication” solution implemented in the previous assignment (Assignment-5) and replace the web services with SSL based authentication.

Thought for 15s

Yes — for **Assignment 6**, the usual Spring answer is to **replace Basic Authentication with HTTPS/SSL**, so the API is accessed over a secure TLS channel instead of sending `Authorization: Basic ...` headers. In Spring Boot, SSL is configured on the embedded server with `server.ssl.*` properties, and Spring Security can be used to require secure channels for requests. Home +1

One important note: **SSL/TLS primarily secures the connection**, while **Basic Auth authenticates a username/password**. So if your faculty says “replace Basic Authentication with SSL based authentication,” the most common interpretation in Spring MVC assignments is: **serve the API only over HTTPS** and remove the Basic Auth requirement. If they mean **true certificate-based client authentication**, that is usually **mutual TLS (mTLS)**, which is a stricter setup than plain HTTPS. Home +1

Assignment 6 answer — Spring REST / Spring MVC with SSL

What changes from Assignment 5

In Assignment 5:

- `GET` was open
- `POST` and `PUT` required Basic Auth

In Assignment 6:

- remove Basic Auth
- enable **HTTPS**
- optionally force all requests to use HTTPS

That means your `/movies` service is now protected by **SSL/TLS** instead of Basic username/password. Spring Boot supports configuring SSL on the web server through SSL properties such as keystore location, password, type, and alias. Home +1

---

1) `pom.xml`

Use this:

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.ap

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.4</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>movies-ssl-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>movies-ssl-demo</name>
    <description>Movie API with SSL</description>

    <properties>
        <java.version>17</java.version>
```

---

2) `MoviesDemoApplication.java`

Java

```java
package com.example.moviesdemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MoviesDemoApplication {

    public static void main(String[] args) {
```

```
        SpringApplication.run(MoviesDemoApplication.class, args);
    }
}
```

3) Movie.java

Java

```java
package com.example.moviesdemo.model;

public class Movie {

    private String movieID;
    private String movieName;
    private String leadActor;
    private String leadActress;
    private String movieDirector;
    private double totalCollection;

    public Movie() {
    }

    public Movie(String movieID, String movieName, String leadActor, String lea
                 String movieDirector, double totalCollection) {
        this.movieID = movieID;
        this.movieName = movieName;
        this.leadActor = leadActor;
        this.leadActress = leadActress;
        this.movieDirector = movieDirector;
        this.totalCollection = totalCollection;
```

4) MovieController.java

Java

```java
package com.example.moviesdemo.controller;

import com.example.moviesdemo.model.Movie;
import jakarta.annotation.PostConstruct;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
```

```
@RestController
@RequestMapping("/movies")
public class MovieController {

    private final Map<String, Movie> movies = new HashMap<>();

    @PostConstruct
    public void init() {
        movies.put("M101", new Movie("M101", "Inception", "Leonardo DiCaprio",
```

---

5) `SecurityConfig.java`

This replaces the Basic Auth setup.

It does **not** use `.httpBasic()`.

It only forces HTTPS.

Java

```
package com.example.moviesdemo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .anyRequest().permitAll()
            )
            .requiresChannel(channel -> channel
                .anyRequest().requiresSecure()
            );
```

What this does

- removes Basic login challenge
- allows requests without username/password
- but **requires HTTPS**

- if a request comes by plain HTTP, Spring Security treats it as insecure and you can configure redirect/secure-only access around that policy. Spring Security's servlet security is filter-based, and `requiresSecure()` is the standard way to require a secure channel. Home +1

---

6) `application.properties`

properties

```properties
spring.application.name=movies-ssl-demo

server.port=8443
server.ssl.enabled=true
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat
```

Spring Boot supports enabling SSL on the embedded server and reading the key material from a keystore, including typical settings for keystore path, password, type, and key alias. Home +1

---

7) Create the SSL certificate / keystore

Put a keystore file in:

```
src/main/resources/keystore.p12
```

You can generate it with Java `keytool`:

Bash

```bash
keytool -genkeypair -alias tomcat -keyalg RSA -keysize 2048 -storetype PKCS12 -ke
```

Then move `keystore.p12` into `src/main/resources`. Spring Boot can use a keystore file as the source of server SSL certificates. Home +1

When it asks for password, use:

```
changeit
```

You can keep the same value as in `application.properties`.

---

8) How to run it

Run the Spring Boot app.

Now your API will run on:

```
https://localhost:8443/movies
```

instead of:

```
http://localhost:8080/movies
```

Using HTTPS on port 8443 is a common secure-server setup in Spring Boot examples and SSL configuration patterns. Home +1

---

9) Postman testing steps

GET all movies

Method:

```
GET
```

URL:

```
https://localhost:8443/movies
```

Click **Send**.

Because this is a self-signed certificate, Postman may warn about SSL certificate validation. For local testing, many students temporarily turn off **SSL certificate verification** in Postman settings or trust the certificate locally. That warning is expected with self-signed local certs. Spring Boot itself is correctly serving HTTPS once the keystore is configured.

Home

GET movie by ID

Method:

```
GET
```

URL:

```
https://localhost:8443/movies/M101
```

POST add movie

Method:

```
POST
```

URL:

```
https://localhost:8443/movies
```

Body → raw → JSON

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 2900.0
}
```

No Basic Auth tab is needed now, because the transport itself is secured by HTTPS.

PUT update movie

Method:

```
PUT
```

URL:

```
https://localhost:8443/movies/M103
```

Body → raw → JSON

JSON

```
{
  "movieID": "M103",
  "movieName": "Avatar Updated",
  "leadActor": "Sam Worthington",
  "leadActress": "Zoe Saldana",
  "movieDirector": "James Cameron",
  "totalCollection": 3000.0
}
```

DELETE

Method:

```
DELETE
```

URL:

```
https://localhost:8443/movies/M103
```

Response:

```
DELETE method is not supported
```

Line-by-line explanation

`server.port=8443`

Runs the server on secure HTTPS port `8443`.

`server.ssl.enabled=true`

Turns on SSL for the embedded server.

`server.ssl.key-store=classpath:keystore.p12`

Tells Spring Boot where the certificate keystore is.

`server.ssl.key-store-password=changeit`

Password to open the keystore.

`server.ssl.key-store-type=PKCS12`

Keystore format is PKCS12.

```
server.ssl.key-alias=tomcat
```

Alias name of the certificate key inside the keystore.

These are standard SSL server properties in Spring Boot's SSL support. Home +1

```
requiresChannel(channel -> channel.anyRequest().requiresSecure())
```

This tells Spring Security every request must come over HTTPS.

```
.httpBasic()
```

Not used anymore, because Assignment 6 says replace Basic Authentication.

Spring Security documents HTTP Basic separately; removing it means you are no longer asking clients for username/password via the `Authorization: Basic` header. Home +1

Exam-style short answer

For Assignment 6, I would write it like this:

> The Basic Authentication solution of Assignment 5 is enhanced by enabling SSL/HTTPS in Spring Boot. The service is exposed only over HTTPS using a server certificate stored in a keystore. Spring Security is configured with `requiresSecure()` so that all requests must use a secure channel. Basic Auth is removed, and the API is accessed through URLs like `https://localhost:8443/movies`. Spring Boot supports SSL through `server.ssl.*` configuration properties. Home +1

One honest note

If your faculty literally means "**SSL based authentication" as client certificate authentication**, then the stricter answer is **mTLS**:

- server has a certificate
- client also has a certificate
- server authenticates the client cert

That is different from normal HTTPS. If you want that stricter version, I can write the **mutual TLS (client certificate) Spring Boot answer** too.

Sources

What plug-in need to be installed inside Eclipse to create a keystore.p12 file

Good question — and here's the important part:

👉 **You do NOT need any Eclipse plugin to create** `keystore.p12` **.**

Keystore is created using a **Java tool called** `keytool` , which comes with your **JDK (Java installation)**.

✅ Correct way (No plugin needed)

🔹 Use `keytool` (recommended)

Step 1: Open terminal / command prompt

- Windows → Command Prompt / PowerShell
- Mac/Linux → Terminal

🔹 Step 2: Run this command

```bash
keytool -genkeypair -alias tomcat -keyalg RSA -keysize 2048 -storetype PKCS12 -ke
```

🔹 Step 3: Enter details

It will ask:

```
Enter keystore password:
```

👉 Enter:

```
changeit
```

Then:

```
What is your first and last name?
```

👉 Example:

```
localhost
```

Then press Enter for rest (or fill details).

🔹 Step 4: File created

You will get:

```
keystore.p12
```

👉 Move this file into:

```
src/main/resources/
```

❗ Important

Make sure your `application.properties` matches:

```properties
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat
```

🚫 Plugins (not required)

You might see suggestions like:

- Keystore Explorer plugin
- Eclipse security plugins

👉 These are **optional tools**, not required.

🧠 Simple understanding

Thing	Meaning
keytool	Java tool to create certificates
keystore.p12	file that stores SSL certificate
Eclipse plugin	❌ not needed

⚠️ Common issues

❌ "keytool not recognized"

👉 Fix:

- Add Java JDK `bin` folder to PATH

  OR
- run with full path:

```bash
"C:\Program Files\Java\jdk-17\bin\keytool.exe" ...
```

❌ Wrong password error

Make sure:

```
changeit
```

matches in both:

- command
- application.properties

👍 Final answer (exam style)

No Eclipse plugin is required to create `keystore.p12`. It can be generated using the `keytool` utility provided with the Java JDK. The generated keystore file is then placed in the project resources and configured in `application.properties` for enabling SSL in Spring Boot.

If you want, I can next show:
👉 how to **fix SSL error in Postman ("self-signed certificate")**
👉 or how to **run HTTPS + HTTP together (redirect setup)**

What is the name of your organization? - its asking me so, what to give

That question comes from the `keytool` setup — it's just asking for **certificate details**, not something strict.

What should you enter?

👉 You can safely give **anything meaningful**, for example:

```
Your organization name?
Wipro
```

OR

```
MyCompany
```

OR even:

```
DemoProject
```

What each question means (simple)

When you run `keytool`, it asks:

Question	What to enter
First and Last Name	`localhost` (important for SSL)
Organizational Unit	`IT` or `Development`
Organization Name	`Wipro` or anything

Question	What to enter
City	Your city
State	Your state
Country Code	`IN`

⭐ Most important field

👉 This one matters most:

```
What is your first and last name?
```

You should enter:

```
localhost
```

Why?

- Because your app runs on:

  ```
  https://localhost:8443
  ```

Example (recommended input)

```
First and Last Name: localhost
Organizational Unit: IT
Organization Name: Wipro
City: Bangalore
State: Karnataka
Country Code: IN
```

Important note

👉 These values are **not validated** in your local project.
They are just **certificate metadata**.

Final simple answer

> For “What is the name of your organization?”, you can enter any value like “Wipro” or your company name. It is only used for certificate details and does not affect functionality.

If you want, I can show you:
👉 how to **fix browser/Postman SSL warning (very common issue)**

This error:

> **“Invalid Public Key Security Object File – This file is invalid for use as Personal Information Exchange”**

means your `keystore.p12` file is **not created correctly OR is corrupted OR not actually a valid PKCS12 file**.

🔍 Why this happens

Common reasons:

❌ 1. File not created using `-storetype PKCS12`

If you missed this, Java may create a `.jks` file but you renamed it `.p12`.

👉 That will cause this exact error.

❌ 2. File got corrupted

- Wrong command
- Interrupted creation
- Manual rename

❌ 3. Using wrong tool to open it

If you double-click `.p12`, Windows tries to import it and may fail if format is wrong.

✅ FIX (step-by-step — do this fresh)

🔹 Step 1: Delete old file

Delete:

```
keystore.p12
```

🔹 Step 2: Create again (correct command)

Run this EXACT command:

```bash
keytool -genkeypair -alias tomcat -keyalg RSA -keysize 2048 -storetype PKCS12 -ke
```

🔹 Step 3: Enter details

Use:

```
Password: changeit
First and Last Name: localhost    ✅ IMPORTANT
Organization: Wipro (or anything)
```

🔹 Step 4: Verify file

Run:

```bash
keytool -list -keystore keystore.p12 -storetype PKCS12
```

If working, you'll see:

```
Alias name: tomcat
```

👉 If this works → file is valid ✅

🔹 Step 5: Move file

Move to:

```
src/main/resources/
```

🔹 Step 6: Check `application.properties`

```properties
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat
```

⚠️ Important clarification

👉 You do NOT need to open `.p12` file manually.

That Windows error comes because:

- you double-clicked it
- Windows tried to import it
- format was wrong

For Spring Boot:
👉 Just place it in project → Spring will use it automatically

---

✅ Quick checklist

Check	Status
Used `-storetype PKCS12`	✅
Password = changeit	✅
Alias = tomcat	✅
File in resources folder	✅
Not renamed manually	✅

🧠 Simple explanation

- `.p12` = certificate container
- Spring Boot reads it
- You don't open it manually

👍 If still not working

Tell me:

- your full command you used
- your Java version

I'll fix it exactly for you.

```
:\Users\akadhira>keytool -genkeypair -alias tomcat -keyalg R
SA -keysize 2048 -storetype PKCS12 -keystore keystore.p12 -va
lidity 3650
Enter keystore password:

keytool error: java.lang.Exception: Key pair not generated, a
lias <tomcat> already exists
```

That error means the keystore file already exists and already contains an entry with alias `tomcat`.

Why it happened

You ran:

```bash
keytool -genkeypair -alias tomcat ...
```

but inside `keystore.p12`, the alias `tomcat` is already present, so Java refuses to create another one with the same name.

Fix options

Option 1: Delete old `keystore.p12` and create again

This is the easiest.

Step 1

Delete the old file:

```
C:\Users\akadhira\keystore.p12
```

Step 2

Run the command again:

```bash
keytool -genkeypair -alias tomcat -keyalg RSA -keysize 2048 -storetype PKCS12 -ke
```

Option 2: Use a different alias

If you want to keep the existing file, use another alias:

```bash
keytool -genkeypair -alias myssl -keyalg RSA -keysize 2048 -storetype PKCS12 -key
```

Then update `application.properties`:

```properties
```

```
server.ssl.key-alias=myssl
```

Option 3: Check what aliases already exist

Run:

```bash
keytool -list -keystore keystore.p12 -storetype PKCS12
```

It will show the alias names inside the keystore.

Best fix for you

Do this:

1. Delete old file

```
C:\Users\akadhira\keystore.p12
```

2. Recreate it

```bash
keytool -genkeypair -alias tomcat -keyalg RSA -keysize 2048 -storetype PKCS12 -ke
```

3. When asked, enter:

- password: `changeit`
- first and last name: `localhost`

4. Move it into:

```
src/main/resources/
```

One more useful command

To verify it is correct:

Bash

```bash
keytool -list -keystore keystore.p12 -storetype PKCS12
```

You should see alias like:

```
tomcat
```

Important

Do not just rename some old file to `.p12`.

It must be created properly by `keytool`.

Paste the next screen after you run `keytool -list`, and I'll tell you if it's good.

```
Your keystore contains 2 entries

myssl, 28-Mar-2026, PrivateKeyEntry,
Certificate fingerprint (SHA-256): 58:5D:42:A2:ED:CF:D9:B7:70
:61:04:76:2D:32:2D:BA:04:59:23:F6:55:F5:E4:7D:5E:4B:4F:B4:E5:
5A:9D:D7
tomcat, 28-Mar-2026, PrivateKeyEntry,
Certificate fingerprint (SHA-256): 1A:53:BB:8D:A9:D6:52:7D:DF
:3B:95:F7:41:F5:EE:5B:73:9E:FD:77:00:6B:AF:A3:6F:C3:0F:C9:B6:
66:F8:E9
```

how to add this keystore.p12 into my eclipse project in eclipse IDE

Perfect — your keystore is **valid** ✅ (you have 2 aliases: `myssl` and `tomcat`).

Now let's properly **add it into your Eclipse Spring Boot project**.

✅ Step-by-step: Add `keystore.p12` into Eclipse project

🔹 Step 1: Locate your project folder

In Eclipse:

👉 Right-click your project

👉 Click **Properties**

👉 Go to **Resource → Location**

OR simply:

👉 Right-click project → **Show in → File Explorer**

🔹 Step 2: Go to resources folder

Open this path:

```
src/main/resources
```

🔹 Step 3: Copy the file

Copy your file:

```
keystore.p12
```

Paste it inside:

```
src/main/resources/
```

🔹 Step 4: Refresh project in Eclipse

👉 Right-click project
👉 Click **Refresh**

Now you should see:

```
src/main/resources
   └── keystore.p12
```

⚙️ Step 5: Update `application.properties`

Open:

```
src/main/resources/application.properties
```

Add:

```properties
server.port=8443
server.ssl.enabled=true
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat
```

⚠️ IMPORTANT (based on your screenshot)

You have TWO aliases:

```
myssl
tomcat
```

👉 You must choose ONE

If using `tomcat`:

```properties
server.ssl.key-alias=tomcat
```

OR if using `myssl`:

```properties
server.ssl.key-alias=myssl
```

🚀 Step 6: Run your application

👉 Right click:

`DemoApplication.java`

👉 Run As → **Spring Boot App**

🌐 Step 7: Test in browser/Postman

Open:

```
https://localhost:8443/movies
```

⚠️ You will see warning (normal)

Browser/Postman will show:

```
Your connection is not private
```

👉 This is NORMAL (self-signed certificate)

Just:

- click **Advanced**
- proceed

🧠 What you just did

Step	Meaning
keystore.p12	SSL certificate file
resources folder	Spring loads from classpath
server.ssl.*	enables HTTPS
alias	selects certificate inside keystore

🔍 Quick checklist

Check	Status
keystore in resources	✅
password matches	✅
alias correct	✅

Check	Status
port 8443	✅
HTTPS working	✅

❗ Common mistakes

❌ File not in resources

→ Spring cannot find it

❌ Wrong alias

→ Error:

```
Cannot recover key
```

❌ Wrong password

→ App fails to start

❌ Using http instead of https

→ Will not work

👍 Final result

Your API is now running securely on:

```
https://localhost:8443/movies
```

If you want next, I can help you:
👉 fix **"Whitelabel Error Page / SSL not working"**
👉 or explain **difference between Basic Auth vs SSL vs JWT (very important interview topic)**

from where to cope keystore.p12 and paste in src folder

Good question — let's make it very clear so you don't get confused.

📍 Where is your `keystore.p12` now?

From your screenshot, you ran command in:

```
C:\Users\akadhira
```

So your file is here:

```
C:\Users\akadhira\keystore.p12
```

👉 That is the file you need to copy.

📂 Where to paste in Eclipse project?

Inside your project:

```
src/main/resources/
```

✅ Step-by-step (easy way)

🔹 Step 1: Open file location

1. Open **File Explorer**
2. Go to:

```
C:\Users\akadhira
```

3. You will see:

```
keystore.p12
```

🔹 Step 2: Copy file

👉 Right click → **Copy**

🔹 Step 3: Open your project folder

In Eclipse:

👉 Right click your project
👉 Click:

```
Show in → File Explorer
```

🔹 Step 4: Navigate inside project

Open:

```
src → main → resources
```

Full path example:

```
your-project-name/src/main/resources/
```

🔹 Step 5: Paste file

👉 Right click → **Paste**

Now structure becomes:

```
src/main/resources/
   ├── application.properties
   └── keystore.p12    ✅
```

🔹 Step 6: Refresh Eclipse

Back to Eclipse:

👉 Right click project
👉 Click **Refresh**

⚙️ Now confirm in `application.properties`

properties

```properties
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat
```

🚀 Run and test

Start your app and open:

```
https://localhost:8443/movies
```

🧠 Simple understanding

Step	Meaning
keystore created in	`C:\Users\akadhira`
copy from	that location
paste into	`src/main/resources`
Spring reads via	`classpath:`

❗ If you don't see file in Eclipse

👉 Do this:

- Right click project → **Refresh**
- OR press `F5`

If you want, I can check your project structure screenshot and confirm everything is correct 👍